Rings of Risk: Consequence and Reachability in Application-Code Vulnerability Chains

John Pellew john@kolega.ai
Faizan Raza faizan@kolega.ai
Kolega.Dev
October 22, 2026

DRAFT · Working research draft · Subject to revision

Abstract

Even when an application's outer perimeter is secured, weaknesses deeper in its code may combine to reach critical outcomes. A file, session or identifier accepted by one feature can cross into another that grants greater authority. Vulnerability counts and individual severity scores do not show whether such routes exist. The Common Vulnerability Scoring System (CVSS) also distinguishes severity from risk. **Rings of Risk** (RoR) traces these routes and profiles each protected outcome by consequence, entry conditions, distinct supported routes and the controls that could interrupt them. Each finding is represented once in the route map and may contribute to multiple chains that reach different outcomes.

We examine 15 public open-source applications, selectively hardening exposed direct routes in six local forks. Source analysis follows code flaws together with artifacts, victim actions and ordinary processing steps. The remaining source contains potential paths to serious outcomes, including multi-flaw chains. Securing an entry point therefore cannot substitute for independent controls deeper in the application code.

1. Introduction

Securing an application's outer perimeter closes its most visible direct routes to critical harm. Yet lower-trust input may still cross several code boundaries and reach a privileged operation. One flaw can supply an identifier, session or file that another flaw wrongly trusts. The application therefore needs independent controls at each handoff: that is the code-level defence-in-depth problem this paper examines.

By **outer perimeter** we mean exposed application entry points and their prominent direct routes to critical outcomes. Addressing those routes is a specified defensive step, not a claim that every external flaw is gone. Our selective fixes create a specified before-and-after version of that scenario in open-source software (OSS) code pinned to fixed revisions. We then examine routes through the controls that remain.

Consider an application whose vulnerability backlog falls from 1,000 to ten. That is progress, but the ten remaining findings could be confined to minor effects or combine into administrator control, customer-data exposure or server execution. The decisive question is which **protected outcomes remain reachable**.

Defence in depth here means independent controls **within application code**: authentication at entry, authorisation at the sensitive operation, tenant separation, safe file processing and checks before privileged execution. A profile-picture upload becomes consequential when attacker-controlled content reaches a vulnerable parser or worker. The upload, handoff and later operation must form a traceable path; accepting a file alone does not establish one.

Security attention may concentrate on conspicuous functions such as login, payments and customer data, while an ordinary feature supplies the first step of a serious chain. This is a motivation to trace cross-feature paths, not an observed pattern of developer or penetration-test behaviour in our sample.

Agentic tooling makes cross-feature search increasingly practical. Work on exploitation of published vulnerabilities and multi-stage activity reinforces the need to examine paths across application features. A one-finding-at-a-time report can obscure the consequential route those features form together.

This paper contributes an outcome- and capability-based risk model; a method for tracing code flaws together with ordinary or external prerequisites; and a 15-application OSS study that contrasts isolated scores with route inventories and examines routes remaining after selective hardening, including paths that the fixes break. It also visualizes how one finding can feed more than one route and how each route ends at a distinct outcome.

2. Background and Related Work

2.1 What existing measures actually measure

Security teams need different answers from a vulnerability assessment: how severe a flaw is, whether attackers exploit it, whether the affected system matters, and what an attacker can reach through it. These are related but different questions. A single finding can disclose a token that enables a second flaw; conversely, a high-severity finding may need an identity or configuration absent from the application being assessed. A score attached to either finding does not, by itself, describe that route.

CVSS v4 classifies and rates individual vulnerabilities. Its Base score expresses technical severity, while Threat and Environmental metrics let a consumer add threat and deployment context. FIRST, which maintains CVSS, explicitly warns against using Base alone as risk. The same user guide **does** explain how an analyst may score a *known* vulnerability chain by combining exploitability and impact metrics. RoR does not claim that CVSS ignores chains. The difference is that a chained CVSS score starts with an identified chain; it does not discover the application's routes, enumerate alternative routes to distinct outcomes or determine which code fixes interrupt them. Adding individual CVSS numbers is not the CVSS method and does not produce an application-risk score. A CVSS Base score is an ordinal severity ranking, so differences between scores are not measured differences in harm or exploitation probability.

The Common Vulnerabilities and Exposures (CVE) Program gives published vulnerabilities common identifiers; the National Vulnerability Database (NVD) enriches CVE records with CVSS, weakness and affected-product information. These are essential reference data for disclosed vulnerabilities, not a score for unpublished first-party findings or a map of an application's code handoffs.

Exploit Prediction Scoring System (EPSS) estimates the probability that exploitation of a published CVE will be observed in the wild over the next 30 days. FIRST says EPSS is not a complete risk score: it does not include the impact or the particular deployment. It updates daily for published vulnerabilities with CVE identifiers. FIRST also cautions against multiplying EPSS by a CVSS Base score: a calibrated probability times an ordinal severity ranking has no clear interpretation. The US Cybersecurity and Infrastructure Security Agency (CISA) maintains the Known Exploited Vulnerabilities (KEV) catalogue of CVEs with evidence of exploitation. It is an important remediation signal, not a probability. Neither can directly score an unpublished flaw in first-party code, and neither establishes whether two local findings compose. They can still inform a route's urgency when its steps have CVE identifiers.

Combining these signals can make a *published-CVE remediation queue* more manageable. In a study of 28,377 vulnerabilities, Shimizu and Hashimoto report roughly 95% less urgent remediation work while retaining 85.6% coverage of their exploitation reference set. That is a result for their filtering policy and data, not an estimate of how much application risk was removed. It does not reveal routes through unpublished findings in first-party code.

CISA's Stakeholder-Specific Vulnerability Categorization (SSVC) uses a stakeholder-specific decision tree to prioritise vulnerability response, incorporating exploitation and organisational impact. It makes a different decision from CVSS or EPSS: what action to take on a known vulnerability. A source-level route and the protected outcome it reaches could supply evidence to such a decision; RoR is not a replacement for that response policy.

2.2 Application and business context

The Open Worldwide Application Security Project's OWASP Risk Rating Methodology combines estimated likelihood and technical or business impact for a particular risk scenario. It can assess an application-specific chain when an analyst has identified one. RoR contributes a repeatable way to describe *which* code steps and ancillary handoffs reach each outcome before that scenario is rated. OWASP's Top 10 is an awareness list of risk categories, not a score for one application; Application Security Verification Standard (ASVS) specifies verifiable security controls. Both help identify controls to inspect at a chain handoff, but neither is a route inventory. MITRE's Common Weakness Enumeration (CWE) provides a vocabulary for weakness types, while ATT&CK organises observed adversary behaviour. They help name a flaw or step without assigning this application's protected outcome or calculating a route.

The National Institute of Standards and Technology (NIST) describes risk in SP 800-30 through a contextual assessment of threats, vulnerabilities, likelihood and consequences for an organisation. Factor Analysis of Information Risk (FAIR) models the probable frequency and magnitude of loss. These broader frameworks are appropriate when there is evidence to estimate organisational impact and event frequency. This source-only study lacks deployment and loss data. Its reachability profile should therefore be an input to a risk assessment, not a numerical probability or expected-loss claim.

The NIST Cybersecurity Framework (CSF) 2.0 and OWASP Software Assurance Maturity Model (SAMM) address how organisations manage security work and improve application security practices. RoR can supply evidence about specific code paths to those programmes; this study does not assess programme maturity or whether an organisation implements their controls.

2.3 Operational prioritisation and attack paths

Commercial prioritisation goes beyond CVSS Base. Tenable's Vulnerability Priority Rating (VPR) combines technical impact and threat intelligence; Rapid7 Active Risk adds exploit and threat signals to CVSS; Qualys TruRisk aggregates findings with asset criticality and exposure. Developer-facing Snyk prioritises by severity, exploit maturity, fixability and project importance. Its reachable-vulnerability analysis can build a call graph for dependencies. These products can use deployed asset data, threat signals or dependency reachability to prioritise a finding. Our study has no deployed asset inventory or live threat telemetry; its contribution is tracing how findings and ordinary code steps connect to protected application outcomes.

Dependency call reachability can show that an application invokes a vulnerable library. RoR asks a different question: whether first-party code flaws and ordinary steps connect a stated starting actor to a protected outcome. A vendor's priority score and that route profile can be used together.

Attack paths are also established practice. MulVAL derives attacker capabilities from facts and prerequisite rules in a network model. Topological Vulnerability Analysis represents dependencies among exploits and the paths to a network attack goal without listing every path separately. The k-zero-day safety literature asks how many unknown flaws a network path would require; its general network formulation is NP-hard. These works establish why combinations and alternative routes matter. They take host, network and configuration facts as inputs rather than deriving application-code handoffs from source. That complexity result does not describe the cost of RoR's bounded review of known code findings.

Microsoft Security Exposure Management and Tenable Attack Path show routes among assets, identities and exposures and identify choke points for remediation. Wiz's Security Graph similarly connects cloud and AI assets to surface consequential combinations. RoR uses the same prerequisite-to-capability idea to examine handoffs within first-party application code and its ordinary consumers: sessions, files, workflows, parsers, logs and jobs. It does not infer a network or cloud path that the source and stated deployment do not establish.

Industry analyses also use paths to narrow remediation. In a reported customer analysis, Praetorian reduced roughly 500,000 vulnerability findings to 14 endpoints exposed to complete browser exploit and sandbox-escape combinations. This is a vendor case, not a prevalence estimate for application-code chains. Cycode describes lower-severity application findings that can contribute to serious outcomes through intermediate steps. RoR takes the complete route as the unit to review, including the code handoff, starting conditions and protected outcome. Vendor examples cannot validate a proposed route in our corpus; that requires review of the pinned application source.

2.4 Program analysis and agent benchmarks

Code property graphs combine syntax, control flow and data dependencies so queries can locate vulnerable code patterns. IRIS combines a large language model with static analysis to infer taint specifications and detect vulnerabilities across Java repositories. Both help discover flaws; RoR starts with findings and follows the capabilities they grant through other code and ordinary processing. VEX-Bench tests whether agents can assess the exploitability of known vulnerable dependencies in downstream projects. That is a different reachability question from composition among first-party application findings.

Agent exploitation benchmarks show why capability and outcome should be kept distinct. CVE-Bench evaluates agents against known web-application CVEs and reports that its strongest framework resolved up to 13% in that test. ExploitBench grades 16 successive capabilities on 41 bugs in the V8 JavaScript engine, from reaching vulnerable code to arbitrary code execution. AgentCyberRange tests web footholds and subsequent movement in ranges with 156 internal hosts; its best reported system solved 16.1% of web-exploitation tasks and 31.7% of post-exploitation tasks under that evaluation. These figures describe particular agents, tasks and budgets, not the likelihood of any RoR route. RoR characterises source-supported routes from a stated start, including unpublished flaws and non-code handoffs; it does not infer actual agent success from those routes. An exploit benchmark tests whether an agent achieves a specified task.

2.5 Why aggregation needs routes

One reachable critical outcome is already critical. A second route to the same outcome does not double the *harm*, but it can increase the opportunity to reach that harm and require another fix. Route count alone is not a probability: routes may share a step or prerequisite. Administrator control and mass customer-data disclosure are also distinct outcomes despite both being critical. RoR therefore keeps consequence, entry difficulty, independent supported routes and the fixes needed to interrupt them for **each protected**

outcome. This is the gap an isolated severity score, a threat-intelligence signal or a finding total cannot fill by simple addition.

If routes had known, independent success probabilities, we could subtract the product of their failure probabilities from one: $1 - \prod(1 - p_i)$ for route probabilities p_i. But application routes often share a credential, vulnerable operation, victim action or deployment condition, so independence rarely holds. This study has no measured p_i values and does not turn route counts into a probability. It instead reports which outcomes each route reaches, what its entry conditions demand, and which fixes interrupt shared or separate paths. Possible route combinations can multiply while the harm of one compromised outcome remains bounded.

Prior work	Unit of analysis	Relationship to RoR
CVSS, EPSS, KEV and operational priority scores	A flaw, published CVE, or affected asset	Useful severity, threat and context signals; RoR adds source-traced routes to specific outcomes.
MulVAL, Topological Vulnerability Analysis and exposure graphs	Network or asset paths	Supply prerequisite-and-capability reasoning; RoR examines handoffs within application code.
Code property graphs, IRIS and VEX-Bench	Vulnerability discovery or dependency exploitability	Help identify or assess individual flaws; RoR follows interactions among known findings and ordinary steps.
Agent exploitation benchmarks	Agent success on specified tasks	Measure what an agent achieved in a test; RoR profiles what routes the assessed source can support.

3. The Rings of Risk model

3.1 Outcome, capability and transition

For each application, a **context card** identifies important functions and assets before findings are graded. Each protected outcome names the resource, operation, affected population and consequence: for example, reading customer records across tenants or executing code in the application-server process. The execution environment matters; the label "remote code execution" alone does not identify what was compromised.

A **capability** is something an actor can do or know, such as reading a file, supplying a workflow property, holding a role or knowing an object identifier. A **transition** uses stated prerequisites and a mechanism to produce another capability. The mechanism may be a code flaw or ordinary application behaviour; a victim action may also be required. Code claims cite source, while external conditions remain explicit assumptions. This prerequisite-to-capability representation draws on attack-graph reasoning such as MulVAL.

A large language model (LLM) proposes capabilities and transitions; source review tests whether each handoff works under the stated identity, permissions and configuration. A route is **source-supported** when those reviewed transitions connect the stated starting actor to a protected outcome. Only reviewed transitions enter the graph. Deterministic code then derives the outcomes reachable from a declared starting position. Citation existence is checked mechanically, while the source review establishes what the cited code actually does.

3.2 Consequence bands and aggregation

Band	Meaning	Illustrative outcome
B1 Critical	Loss of an essential function or authority	Server-process execution, platform-administrator control or mass credential disclosure
B2 Major	Substantial data or operational harm	Cross-tenant disclosure of customer records

Band	Meaning	Illustrative outcome
B3 Moderate	Meaningful but bounded and recoverable harm	Alteration of one team's non-authoritative records
B4 Limited	Local, readily reversible harm	Disclosure of a version string

B1 Critical is the highest consequence, followed by B2 Major, B3 Moderate, and B4 Limited.

Each band grades a specified outcome, not an application. RoR keeps distinct critical outcomes separate and does not add their bands or count additional routes as additional harm. A highest-band headline can point to the full profile, but cannot replace it.

3.3 Chains and prerequisites

An **attack route** connects a starting actor to a protected outcome. A **chain** has at least two distinct security-relevant transitions: one step supplies an artifact or capability that a separate step uses to cross another trust or authority boundary. At least one step involves a code flaw. A *multi-vulnerability chain* contains two or more distinct code flaws. A *mixed chain* includes a separate, capability-changing handoff through a credential, victim action, document, browser, log, workflow or other ordinary behaviour. We record the full route and the number of code flaws separately. A prerequisite does not automatically become a chain link: SQL injection used to dump a database is one flaw's direct route, even if it takes multiple queries. Uploading a file to trigger a vulnerable parser is likewise not a chain merely because storage and processing are separate operations. Phishing can bridge steps, but must be stated as a condition unless its role and outcome are independently supported.

A consequential chain matters even when a shorter route also exists, and it need not reproduce the outcome of a flaw chosen for repair. After hardening, we ask whether its own steps still connect to its own outcome; a route that requires patched code no longer survives. Each handoff must preserve the actor's capability across environments: browser execution, server execution, continuous-integration (CI) execution and PDF creation are different outcomes unless code connects them.

Code-adjacent handoffs provide the search map for those connections. A file, message, credential or URL may carry attacker influence without itself being a vulnerability. The important event is its later consumption by a different principal or more privileged component. We search for the producer, carrier, consumer, trigger and resulting capability, then test that handoff against source and stated deployment conditions.

Bridge to examine	Evidence needed to admit the handoff
Human and identity: phishing, approval, login, token or invitation	Who acts; how a valid credential or role is obtained; its scope, expiry and receiving operation.
Browser and client: rendered content, localhost request, extension or native app	Viewer action, origin, session and permissions; an extension or client must actually be installed and reached.
Artifact and document: upload, model, archive, backup, image or PDF	Attacker control of bytes; storage and later parser, importer, viewer or worker; the exact processing trigger.
Log, message, cache, search result or agent input	Writer and reader; how stored data becomes a displayed value, leaked secret, tool instruction or privileged action.
Webhook, callback, queue or scheduled job	Producer, service identity, access control, message or URL validation, and the consumer's operation.
Build, plugin and deployment component: CI runner, package, cloud identity or guest	Installation or job trigger; runner or service permissions; any further boundary to server, cloud asset or host.

This is a catalogue of *possible* bridges, not a list of observed attacks. File-upload processing, log consumption, CI script evaluation and agent use of untrusted text illustrate distinct consumer boundaries. Browser content does not inherit a browser extension's permissions; PDF scripting depends on the viewer's controls. Crossing from a browser, viewer, runner or guest into the application server or host requires another supported transition.

3.4 Access ring, reach ring and effort

The **Access Ring (AR)** places an attacker, or a protected asset, by the privilege tier it holds or requires. Rings are numbered from the core, as the consequence bands are, so AR1 is the innermost.

Ring	Name	Position	Example
AR5	Outside	No account or foothold: anonymous, or holding only a lure, a leaked link or an identifier	Anonymous request; enumerated object ID
AR4	Standard user	A low-privilege account, including a self-registered or tenant-member account	Ordinary logged-in session
AR3	Delegated authority	Another principal's authority: an administrator or owner role, or a victim's authenticated session	Administrator session taken through stored script injection
AR2	Control plane	Platform, tenant or organisation administration; secret and key stores; cross-tenant reach	Tenant-wide administration; stored credentials
AR1	Crown jewels	Host or server-process code execution; control of the whole datastore or of the identity root	Server-process execution; arbitrary SQL

A vulnerability finding is an arrow. Its tail is the ring the attacker occupies when exploiting it, and its head is the ring the attacker holds afterwards, so its length is the number of rings it crosses: what the finding gives. A finding that discloses an identifier but confers no authority leaves the attacker in AR5 and enables a later arrow instead of moving the attacker inward. Every arrow ends in a circle. A final outcome is a circle inside the ring whose privilege the outcome puts under attacker control. An intermediate outcome is the position or capability a finding gives on the way, such as an administrator session or a leaked secret, and the next finding starts from that circle, which makes the chain and its hand-offs visible. A finding used by several routes is drawn once.

The ring and the consequence band are different measures. The band grades the harm of an outcome; the ring says where the attacker stands. A B1 Critical account takeover can end in AR3, whereas server-process execution ends in AR1. Rings are assigned from the reviewed capability and outcome of each step under a stated rule, not from CVSS: an outcome takes the ring of the privilege needed to perform that function legitimately, and a capability takes the ring of the authority it confers. Judgement calls are marked in the classification table. An enabling step that puts an account, credential or a victim's session within the attacker's reach, such as open self-registration, a phished password or a lure that makes a privileged user open attacker content, is drawn as a dashed arrow into the ring it reaches (AR4 for an ordinary account, AR3 for a privileged victim), and the first code arrow starts from that ring. A configuration, file or missing control gives the attacker no position of its own, so such a route starts in AR5. A first code flaw whose starting position the ledger does not record is drawn entering from outside the rings, and no starting ring is assumed.

The **Reach Ring (RR)** classifies a finding by what it can reach from a declared starting position. The consequence band grades an outcome; RR grades a finding's role in reaching it. In ring terms, an RR1 or RR2 finding is a single arrow from the declared start to a B1 Critical or B2 Major outcome, whereas an RR3 finding is an arrow that ends short of the outcome and contributes only as an earlier segment of a longer path. RR4 Bounded reach describes a finding; B1 Critical describes an outcome.

Level	Name	Rule in the declared scenario
RR1	Critical reach	The finding can reach B1 Critical without another code flaw
RR2	Major reach	It can reach B2 Major without another code flaw, but not B1 Critical
RR3	Chain-dependent	It contributes to B2 Major or B1 Critical only with another code flaw
RR4	Bounded reach	Its reachable outcomes are B4 Limited or B3 Moderate only
RR5	No established reach	No outcome is established from the declared start

Victim actions, credentials and configuration are recorded alongside each RR assignment. A route involving one code flaw and a victim action can still be RR1 Critical reach or RR2 Major reach; the action affects entry difficulty, not the code-flaw count. A finding at either level can also carry a chain flag. RR5 No established reach means that reach was not established from this starting position.

Figure 1 introduces the visual notation. The rings are drawn uncurled, as consecutive columns from AR5 at the left to AR1 at the right, so that the distance an arrow travels reads as the number of columns it crosses. Vulnerability findings are arrows that cross rings, protected outcomes are circles, and a shared finding is drawn once with later arrows starting where it ends. Dashed links mark supporting conditions such as credentials, victim actions or configuration. The example is schematic; it does not depict a corpus application.

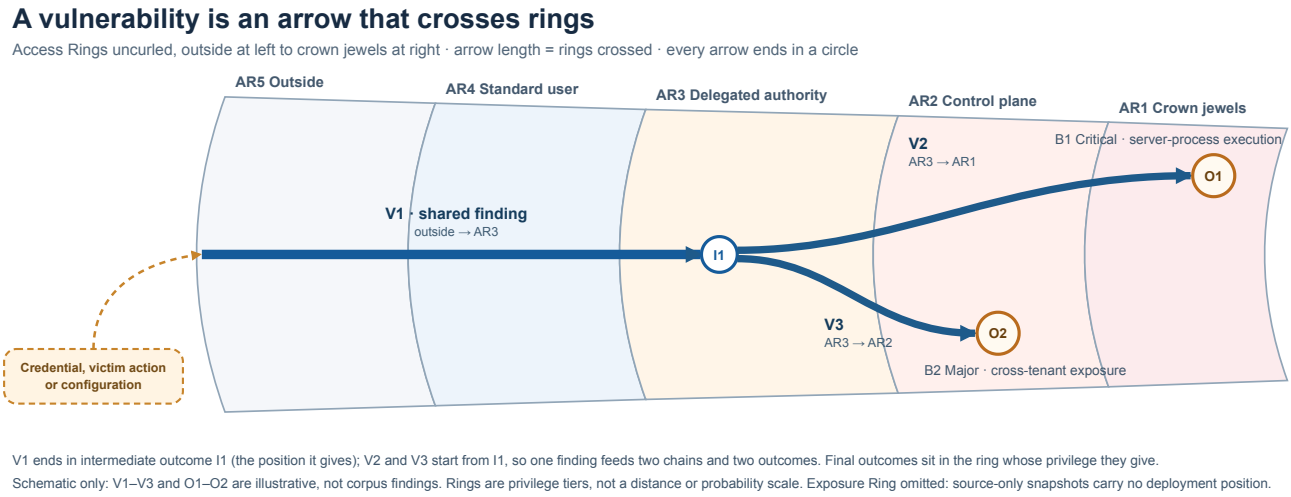


Figure 1. The rings are drawn uncurled, from the outermost (AR5, left) to the crown jewels (AR1, right). Vulnerabilities are arrows that cross Access Rings and end in circles. V1 is a shared finding that takes the attacker from AR5 to AR3 and ends in intermediate outcome I1; V2 and V3 start from I1 and reach different final outcomes. The example is schematic; V1–V3 are not corpus findings.

For each outcome, RoR records the demands of every supported route: starting access, victim action, configuration, code-flaw count and evidence state. It then identifies the least demanding routes. When one needs a victim click and another needs an extra code flaw, neither is automatically easier; both remain visible.

An **Exposure Ring** can separately describe where a component sits in a deployment. This source-only study has no consistent entry-point or network policy data, so it records application entry conditions without assigning deployment exposure levels.

3.5 Outcome profiles and known-route closure

RoR reports a **structured profile for each protected outcome**:

Field	Question answered
Consequence	What exact resource, operation and population are affected, and at which band from B1 Critical to B4 Limited?
Entry difficulty	What access and conditions does each supported route require, and which routes are least demanding?
Route count	How many distinct source-supported routes reach this outcome? Candidate records are displayed separately.
Known-route closure	What is the smallest feasible set of code fixes or controls that interrupts every *known supported route* to this outcome?

For each outcome, the profile contains its consequence band, the entry requirements and least demanding routes, the number of distinct supported routes, and the minimum number of fixes needed to interrupt known routes. The profile keeps different kinds of evidence visible instead of adding them into an opaque score. Outcomes at the same band can have different entry conditions and remediation work. Route count and entry difficulty indicate attack opportunity; they are not a calibrated probability of exploitation.

Closure depends on shared mechanisms. Twenty variants through one vulnerable operation that consumes attacker input may require one fix; three independent routes may require three. The field covers **known supported routes**. A compact profile might read: "critical customer-data disclosure; ordinary-account entry; three supported routes; two fixes to close the known routes." The underlying paths and fixes remain available for review.

The current OSS ledger is an input to this profile, not a completed outcome profile. Its records include overlapping variants and unreviewed handoffs. Distinct supported-route and closure counts require those records to be adjudicated against specified outcomes.

3.6 Found and residual risk

Here, **ground truth** means the study's reference inventory of findings used to evaluate scanner alerts. **Found risk** describes outcomes reachable through findings reported by a scan and can guide remediation without ground truth. **Scanner residual risk** is a benchmark quantity: after treating matched ground-truth findings as fixed, it asks which outcomes remain reachable through missed findings. **Post-hardening residual reach**, used in this study, instead inspects routes remaining after documented fixes to pinned source. A live system's unknown misses cannot be measured directly, so these quantities answer different questions.

4. Study design

4.1 Corpus and route inventory

We compare the available CVSS scores for ground-truth findings with candidate route records across **all 15** pinned applications. Their simple score sums are shown only to test what an additive finding-level summary conveys; they are not CVSS risk scores. The six locally changed forks provide the narrower before/after comparison. The nine unchanged repositories supply baseline source data, not observed results of a fix budget.

The 15 applications (pseudonymised). Application names, versions and vulnerability identifiers are withheld to respect coordinated vulnerability disclosure; each application is described by what it does, and its size and adoption are given as coarse bands.

Application	What it does	Size (lines of code)	Adoption (GitHub stars)	Contributors
Aspen	Workflow-automation platform	>500k	>50k	750+
Basalt	Identity and access management server	>500k	10-50k	250-750
Cobalt	Self-hosted chat front end for language models	100-250k	>50k	250-750
Dune	Machine-learning experiment and model-management platform	100-250k	10-50k	250-750
Ember	Mobile and web backend service	100-250k	10-50k	250-750
Fjord	Software-translation management platform	100-250k	<10k	<250
Garnet	Publishing and membership platform	100-250k	>50k	750+
Harbor	Workflow-orchestration platform	250-500k	10-50k	750+
Indigo	Visual builder for language-model agents and pipelines	100-250k	>50k	250-750
Juniper	Download manager with a web interface	<100k	<10k	<250
Kestrel	Enterprise wiki and application platform	250-500k	<10k	<250
Lumen	Web-interface toolkit for machine-learning demos	<100k	10-50k	<250
Marble	Build-automation server	100-250k	10-50k	750+
Nimbus	Visual builder for language-model workflows	100-250k	>50k	250-750
Onyx	Low-code builder for internal tools	250-500k	10-50k	<250

Size is non-test source code in the pinned snapshot, excluding generated, translation and data files. Stars and contributors are those of the public upstream project in October 2026. Clone, download and user counts are not available for third-party projects and are not used.

The chain ledgers contain 801 proposed pairs of steps for all 15 applications. A pair is a route proposal, not necessarily a complete chain to an outcome. We retain non-rejected routes labelled B2 Major or B1 Critical, remove routes requiring a patched flaw in the relevant modified fork, and collapse identical ordered first-step/second-step/outcome records. We assess whether each handoff exists regardless of any shorter route. The result is **246 route records carrying source references**: 123 beginning with a code flaw, 123 with an ancillary step, and 178 labelled with a B1 Critical outcome. An ancillary step is a condition such as a credential, victim action or ordinary application operation, rather than another code flaw. Nineteen records have a stale or imprecise source reference. These counts describe the review inventory, not 246 distinct established chains.

We inspect source, fixes and configuration without running applications. A route is **source-supported under conditions** when the code establishes its steps and handoffs, and any victim or deployment conditions are stated. A record with citations but an unverified capability transfer remains a *candidate*; contradicted or patched paths are removed. This source standard does not claim that an exploit was executed.

To identify longer paths, we also join a route's resulting finding to another route that begins with that finding, then check whether the first step supplies the second step's required capability, role or token. This exact-ID pass produced **54 joined path records** across four applications (Onyx, Garnet, Basalt and Cobalt). They are overlapping graph paths, not 54 distinct chains: source review rejects two joins in Cobalt because the reviewed code rejects the empty credential they relied on, and some apparent Basalt starts do not provide the registration credential required by the next step. The retained longer paths and their conditions are described in §5.2; the composed-path register records every generated path.

4.2 Selective interventions

In six local forks, we fix selected direct routes from exposed application entry points to serious outcomes, then examine the routes still present in the code. Each intervention is tied to a specific flaw and fix, with identifiers withheld here. This models outer-perimeter hardening at the named routes without claiming that every exposed flaw was closed.

Fork	Local intervention	CVSS sum: pinned → minus targeted findings	Retained B2 Major/B1 Critical records	Using two code flaws	Records labelled B1 Critical
Ember	Upstream fixes for three findings	562.4 → 533.2	15	3	7
Garnet	Upstream fix for one finding	318.6 → 309.2	21	18	18
Indigo	Upstream fixes for two findings	197.2 → 178.3	7	6	2
Juniper	Locally required an operator-supplied initial administrator password	152.4 → 152.4	8	3	7
Lumen	Upstream fixes for three findings	150.3 → 127.7	5	0	1
Onyx	Upstream fix for one finding	415.3 → 406.3	11	9	5
Total	**Ten finding fixes plus one credential hardening**	**1,796.2 → 1,707.1**	**67**	**39**	**40**

The six modified forks retain 67 route records, 39 involving two code flaws. Several share steps or outcomes, so the count is not an aggregate harm score. The fixes also eliminate some proposed chains: Lumen's file-read fix breaks seven recorded pairs that depended on that code, including four proposals labelled B1 Critical. The intervention therefore reveals both surviving routes and routes a shared fix closes.

4.3 Route assessment

For each route, we identify the lower-trust producer and carrier, the exact consumer, the trigger that brings them together, and the capability gained at that boundary. We then record the starting actor, cited code steps, credential or configuration conditions, protected outcome and execution environment. The source review checks that the produced artifact, identity or value can actually reach the named consumer with the claimed authority. We distinguish a separate capability-changing handoff from the ordinary delivery, trigger or consequence of one flaw. We also check whether a selected fix removes a required step. The question is whether **this route's steps compose** to reach **this outcome**, regardless of another available route.

5. Findings

5.1 The full 15-application view

All 15 applications enter the cross-sectional comparison below. Six local forks have documented changes, permitting a before/after route check; the other nine remain pinned baselines. Every application has route records labelled B2 Major or B1 Critical, but those records are not distinct harms or a completed RoR score. The CVSS column deliberately shows what simple addition yields; it is **not** a CVSS-sanctioned measure of application risk. Its sums describe the pinned ground-truth inventories, while the route columns use modified forks in six cases and pinned baselines in nine. The figures are descriptive inputs, not a ranking of these applications.

Application	Ground-truth findings	Sum of available CVSS scores*	Local state	B2 Major/B1 Critical route records	Records labelled B1 Critical
Aspen	125	917.5	Pinned	13	5
Basalt	66	381.5	Pinned	49	42
Cobalt	60	386.9	Pinned	8	5
Dune	30	249.8	Pinned	11	11
Ember	81	562.4	Selectively hardened	15	7
Fjord	33	171.4	Pinned	13	8
Garnet	52	318.6	Selectively hardened	21	18
Harbor	32	203.0	Pinned	5	1
Indigo	25	197.2	Selectively hardened	7	2
Juniper	22	152.4	Credential hardened	8	7
Kestrel	153	1,276.6	Pinned	63	54
Lumen	22	150.3	Selectively hardened	5	1
Marble	30	170.6	Pinned	8	6
Nimbus	92	719.3	Pinned	9	6
Onyx	56	415.3	Selectively hardened	11	5
Total	**879**	**6,272.8**	**Six changed, nine unchanged**	**246**	**178**

*Three ground-truth findings lack a numeric CVSS score (two in Onyx, one in Dune). The sum uses the other 876 scores. Route records vary in review depth and may share steps or outcomes, so their count is not an alternative risk score. Only the six changed forks support an observed before/after comparison; placing hypothetical fixes in the nine baselines would require a separate intervention and route review.

Dune's ledger contains 13 conditional victim- or untrusted-artifact routes. Eleven are provisionally labelled B1 Critical and enter the table; two browser-rendered routes are B3 Moderate and do not. Nine routes labelled B1 Critical have source-supported processing handoffs under stated artifact or victim conditions. The parameter-injection and template-rendering routes remain candidates because attacker control of the later input is unconfirmed. No Dune route in this ledger links two code flaws, and its pinned fork has not been selectively hardened. The ledger is a scoped source review of victim/artifact handoffs, not an exhaustive enumeration of possible pairs.

Route records cannot be read as independent risks. Kestrel's 63 include victim/configuration variants and repeated sinks; Basalt's 49 include specific identities and integrations. Their eventual outcome profiles must group shared routes, retain distinct outcomes and state each route's entry conditions. A shorter route changes immediate fix priority without erasing a deeper path.

5.2 Source-supported chain patterns

Artifact to processing. Dune's pinned fork provides a clear reason to include code-adjacent steps: an untrusted model or pipeline artifact must be selected and later loaded or rendered by a user or worker. The ledger records eleven artifact- or parameter-to-processing routes labelled B1 Critical, nine with source-supported handoffs under that condition. Two more need confirmation that attacker input reaches the later command or template. These paths end in the loading or rendering process; they do not show that the public server automatically executes an uploaded file.

Lower-trust content to privileged processing. In Juniper's credential-hardened fork, an account holding a low-privilege permission can place content in a view that an administrator later opens. When the administrator opens it, the cited rendering flaw can run script in that administrator's browser. A separate session-storage and upload weakness is proposed as a later step toward server execution; that browser-to-server handoff still needs source confirmation. The password change leaves the cited code steps in place. The route illustrates why the browser and server capabilities must be traced separately.

Identity and permission handoff. Fjord, an unchanged baseline, shows how a low-rated permission defect could open a deeper operation. In the pinned source, ordinary permission collection skips a group that enforces two-factor authentication, while global permission collection uses that group's roles, so a project-creation permission is granted more broadly than intended. A separate backup-restore flaw can write content into a version-control component; later processing of that content is the proposed server-execution step. This conditional route requires a privileged password, the specified group policy and that later processing trigger. It is not an anonymous-entry route.

Privileged browser to server. Garnet's local fix closes the selected anonymous content-API route. Remaining records propose a different path: attacker-influenced embedded content reaches an administrator's browser, then a browser/session capability is used toward a template-helper path to server execution. The relevant code-step files remain after the fix. An obsolete fixture citation and an incomplete permission handoff prevent those records from establishing the full server outcome yet; they identify a concrete cross-boundary path for adjudication.

Routes a fix actually breaks. Four proposed Lumen file-read chains labelled B1 Critical used one file-read finding as their second step. The local fix removes that step, so those chains do not survive in the modified fork. Other Lumen records involve a victim browser reaching localhost or a built-in tunnel-sharing feature. The browser, tunnel and CI runner remain distinct environments; the source does not connect them into one server-execution chain.

Other proposed routes use an identifier disclosed before a gated operation in Nimbus or Indigo, a log or import error as an information carrier in Harbor, an identity-provider flow requiring a real provider account in Aspen, or content rendered in a browser before document creation in Cobalt. The catalogue directs attention to the next consumer in each case. Reading a log does not execute it; exporting a document does not run embedded code. None of the reviewed source routes establishes a malicious browser extension or virtual-machine escape.

Longer compositions and shared findings. Joining source-reviewed route segments shows why a finding list alone hides route structure. In Onyx, an identity-information disclosure can feed a single-sign-on account takeover; the resulting global account session can reach a separate licence-control flaw. This is a conditional three-flaw path to account and licence control. In Garnet, a staff password-hash exposure can feed a device-verification bypass; the resulting staff session can then reach separate administrator or staff operations. Four recorded hash-source findings and three downstream outcomes produce 12 ledger combinations, but they share mechanisms and are not 12 independent chain types. In Basalt, a valid initial-access token can open a client-management flaw, a victim administrator's visit can create an admin session, and separate tenant-permission and component/directory flaws can extend that session to server execution. The route depends on the token, victim action, role and component configuration.

Long end-to-end paths also include non-code stages. In Fjord, a compromised privileged password, a group two-factor/global-permission flaw, project creation and backup/version-control processing form a conditional path involving two code flaws and a later worker operation. In Juniper, low-privilege content reaches an administrator's browser, then a separate server-side flaw; the victim's view is the bridge. These

paths have more attack stages than code flaws. A finding shared by several paths is counted once in the map and appears in every route that uses it.

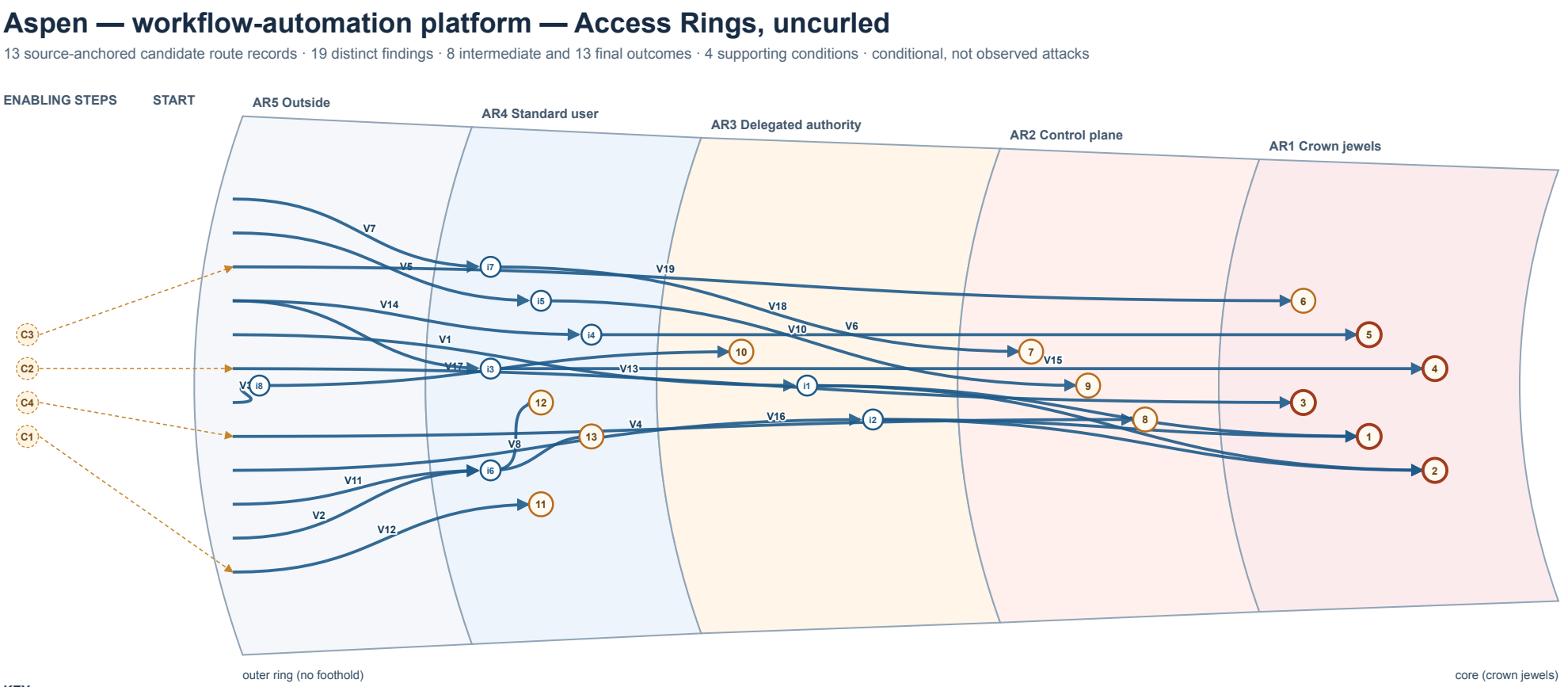
The 54 joined records are a mechanical search result, not the study's chain count. Exact finding-ID joins find only links represented in the ledgers; capability-based joins can also connect a credential, role, artifact or ordinary operation to another vulnerable feature. The composed-path register lists all 54 generated paths, including rejected joins, and the full route register lists all 246 underlying route records. The present review supports longer candidate families; it does not claim to have enumerated every possible capability-based composition in all 15 repositories.

5.3 Repository route maps

Figures 2–16 show one Access Ring map for each pinned application, with the rings drawn uncurled from AR5 (left) to AR1 (right). Vulnerabilities are labelled V1, V2, … and each map's legend gives the audit identifier for every code. Each map includes every source-anchored route record in the current audit. A vulnerability is one labelled arrow from the ring the attacker holds to the ring it reaches, so its length shows what it gives. Every arrow ends in a circle: a final outcome in the ring whose privilege it puts under attacker control, or an intermediate outcome (the position or capability a finding gives), and the next arrow starts from that circle. A finding used by several routes is drawn once. Dashed amber steps are supporting conditions (credentials, victim actions, configuration, files) and are not counted as vulnerabilities.

Rings are assigned from the reviewed capability and outcome of each step under the rule in §3.4; judgement calls are marked in the classification table. Where the ledger does not record a starting position for a first code flaw, its arrow enters from outside the rings and no starting ring is assumed. Exposure Rings are omitted because the snapshots carry no deployment position. Section 6.4 reads what the maps show. Full source gates and conditional assumptions remain in the route register and the route audit. All supporting files are in the project repository.

Figure 2. Aspen, workflow-automation platform: all 13 source-anchored route records.



KEY

- Code vulnerability (V#). Tail = ring held; head = ring reached; length = rings crossed.
- i1 — Intermediate outcome (i#): position or capability gained; the next vulnerability starts here.
- 1 — Final outcome, B1 Critical (#).
- 2 — Final outcome, B2 Major (#).
- Enabling step that puts an account, credential or a victim's session within reach: moves the attacker into that ring.
- C1 — Enabling step that gives no position (configuration, file, missing control): sets conditions for the next vulnerability.
- Dotted grey = start position not recorded in the ledger (arrow enters from the left of the rings).

Vulnerabilities (V# = code in the audit)

V1 · Aspen-F01
V2 · Aspen-F02
V3 · Aspen-F03
V4 · Aspen-F04
V5 · Aspen-F05
V6 · Aspen-F06
V7 · Aspen-F07
V8 · Aspen-F08
V9 · Aspen-F09
V10 · Aspen-F10
V11 · Aspen-F11
V12 · Aspen-F12
V13 · Aspen-F13
V14 · Aspen-F14
V15 · Aspen-F15
V16 · Aspen-F16
V17 · Aspen-F17
V18 · Aspen-F18
V19 · Aspen-F19

Final outcomes (red = B1 Critical, amber = B2 Major)

1 · AR1 · B1 · server-side code execution (a)
2 · AR1 · B1 · server-side code execution (b)
3 · AR1 · B1 · server-side code execution (c)
4 · AR1 · B1 · server-side code execution (d)
5 · AR1 · B1 · server-side code execution (e)
6 · AR1 · B2 · arbitrary file write on the server (a)
7 · AR2 · B2 · arbitrary database access
8 · AR2 · B2 · arbitrary file write on the server (b)
9 · AR2 · B2 · secret or credential disclosure
10 · AR3 · B2 · unauthorised modification or deletion
11 · AR4 · B2 · data disclosure beyond the caller's permissions (a)
12 · AR4 · B2 · data disclosure beyond the caller's permissions (b)
13 · AR4 · B2 · data disclosure beyond the caller's permissions (c)

Intermediate outcomes (position or capability a finding gives; the next finding starts here)

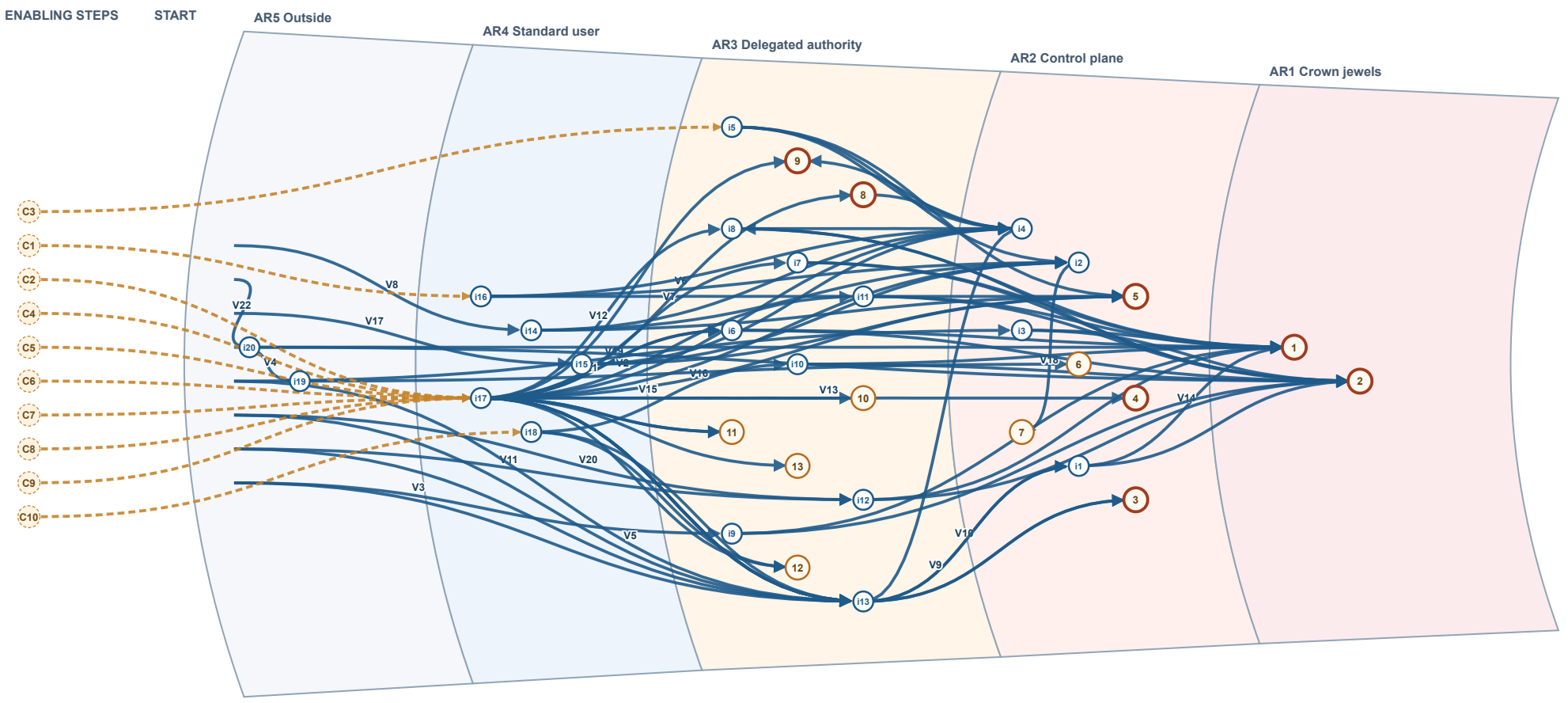
i1 · AR3 · administrator session obtained through an indirect route
i2 · AR3 · administrator or staff session obtained
i3 · AR4 · standard-user account or session obtained (a)
i4 · AR4 · standard-user account or session obtained (b)
i5 · AR4 · another user's credential reused
i6 · AR4 · attacker-controlled data written (a)
i7 · AR4 · attacker-controlled data written (b)
i8 · AR5 · identifier or metadata disclosed

Enabling steps (not vulnerabilities)

C1 · publicly exposed endpoint or workflow (a)
C2 · publicly exposed endpoint or workflow (b)
C3 · publicly exposed endpoint or workflow (c)
C4 · publicly exposed endpoint or workflow (d)

Rings are Access Rings (§3.4), drawn uncurled: outermost (AR5) at left, crown jewels (AR1) at right. Assignments follow the rule in §3.4; judgement calls are marked in the classification table. Exposure Ring unavailable (no deployment data).

Figure 3. Basalt, identity and access management server: all 49 source-anchored route records.



KEY

- Code vulnerability (V#). Tail = ring held; head = ring reached; length = rings crossed.
- i1 — Intermediate outcome (i#): position or capability gained; the next vulnerability starts here.
- 1 — Final outcome, B1 Critical (#).
- 2 — Final outcome, B2 Major (#).
- Enabling step that puts an account, credential or a victim's session within reach: moves the attacker into that ring.
- C1 — Enabling step that gives no position (configuration, file, missing control): sets conditions for the next vulnerability.
- Dotted grey = start position not recorded in the ledger (arrow enters from the left of the rings).

Vulnerabilities (V# = code in the audit)

V1 · Basalt-F01
V2 · Basalt-F02
V3 · Basalt-F03
V4 · Basalt-F04
V5 · Basalt-F05
V6 · Basalt-F06
V7 · Basalt-F07
V8 · Basalt-F08
V9 · Basalt-F09
V10 · Basalt-F10
V11 · Basalt-F11
V12 · Basalt-F12
V13 · Basalt-F13
V14 · Basalt-F14
V15 · Basalt-F15
V16 · Basalt-F16
V17 · Basalt-F17
V18 · Basalt-F18
V19 · Basalt-F19
V20 · Basalt-F20
V21 · Basalt-F21
V22 · Basalt-F22

Final outcomes (red = B1 Critical, amber = B2 Major)

1 · AR1 · B1 · server-side code execution (a)
2 · AR1 · B1 · server-side code execution (b)
3 · AR2 · B1 · platform-level administration taken over (a)
4 · AR2 · B1 · trust or integrity compromise of a shared component
5 · AR2 · B1 · platform-level administration taken over (b)
6 · AR2 · B2 · data disclosure beyond the caller's permissions
7 · AR2 · B2 · protected outcome reached
8 · AR3 · B1 · administrator or account session taken over (b)
9 · AR3 · B1 · administrator or account session taken over (c)
10 · AR3 · B2 · administrator or account session taken over (a)
11 · AR3 · B2 · administrator or account session taken over (d)
12 · AR3 · B2 · administrator or account session taken over (e)
13 · AR3 · B2 · administrator or account session taken over (f)

Intermediate outcomes (position or capability a finding gives; the next finding starts here)

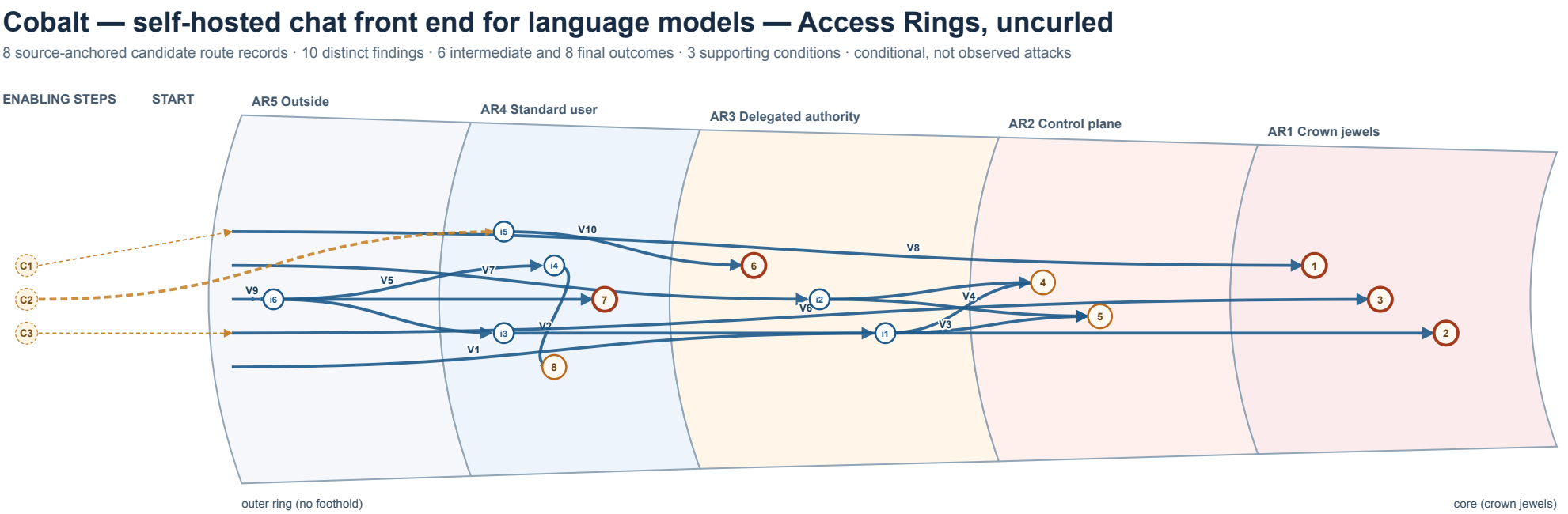
i1 · AR2 · platform-level administration obtained (a)
i2 · AR2 · secret or credential read
i3 · AR2 · platform-level administration obtained (b)
i4 · AR2 · client-management authority obtained
i5 · AR3 · privileged credential held
i6 · AR3 · administrator session obtained through an indirect route (a)
i7 · AR3 · administrator session obtained through an indirect route (b)
i8 · AR3 · administrator session obtained through an indirect route (c)
i9 · AR3 · administrator session obtained through an indirect route (d)
i10 · AR3 · administrator session obtained through an indirect route (e)
i11 · AR3 · administrator session obtained through an indirect route (f)
i12 · AR3 · administrator or staff session obtained
i13 · AR3 · administrator session obtained through an indirect route (g)
i14 · AR4 · standard-user account or session obtained (a)
i15 · AR4 · standard-user account or session obtained (b)
i16 · AR4 · standard-user account available (open registration or granted permission)
i17 · AR4 · standard-user account or credential held
i18 · AR4 · ordinary user's session reached by attacker content
i19 · AR5 · identifier or metadata disclosed (a)
i20 · AR5 · identifier or metadata disclosed (b)

Enabling steps (not vulnerabilities)

C1 · open registration or permissive default
C2 · credential, token or account obtained (phishing, issuance or reuse) (a)
C3 · privileged credential or token obtained
C4 · credential, token or account obtained (phishing, issuance or reuse) (b)
C5 · credential, token or account obtained (phishing, issuance or reuse) (c)
C6 · credential, token or account obtained (phishing, issuance or reuse) (d)
C7 · credential, token or account obtained (phishing, issuance or reuse) (e)
C8 · credential, token or account obtained (phishing, issuance or reuse) (f)
C9 · credential, token or account obtained (phishing, issuance or reuse) (g)
C10 · victim opens attacker-influenced content

Rings are Access Rings (§3.4), drawn uncurled: outermost (AR5) at left, crown jewels (AR1) at right. Assignments follow the rule in §3.4; judgement calls are marked in the classification table. Exposure Ring unavailable (no deployment data).

Figure 4. Cobalt, self-hosted chat front end for language models: all 8 source-anchored route records.



KEY

- Code vulnerability (V#). Tail = ring held; head = ring reached; length = rings crossed.
- i1 — Intermediate outcome (i#): position or capability gained; the next vulnerability starts here.
- 1 — Final outcome, B1 Critical (#).
- 2 — Final outcome, B2 Major (#).
- Enabling step that puts an account, credential or a victim's session within reach: moves the attacker into that ring.
- C1 — Enabling step that gives no position (configuration, file, missing control): sets conditions for the next vulnerability.
- Dotted grey = start position not recorded in the ledger (arrow enters from the left of the rings).

Vulnerabilities (V# = code in the audit)

V1 · Cobalt-F01
V2 · Cobalt-F02
V3 · Cobalt-F03
V4 · Cobalt-F04
V5 · Cobalt-F05
V6 · Cobalt-F06
V7 · Cobalt-F07
V8 · Cobalt-F08
V9 · Cobalt-F09
V10 · Cobalt-F10

Final outcomes (red = B1 Critical, amber = B2 Major)

1 · AR1 · B1 · server-side code execution (a)
2 · AR1 · B1 · server-side code execution (c)
3 · AR1 · B1 · server-side code execution (d)
4 · AR2 · B2 · trust or integrity compromise of a shared component (a)
5 · AR2 · B2 · trust or integrity compromise of a shared component (b)
6 · AR3 · B1 · server-side code execution (b)
7 · AR4 · B1 · administrator or account session taken over
8 · AR4 · B2 · data disclosure beyond the caller's permissions

Intermediate outcomes (position or capability a finding gives; the next finding starts here)

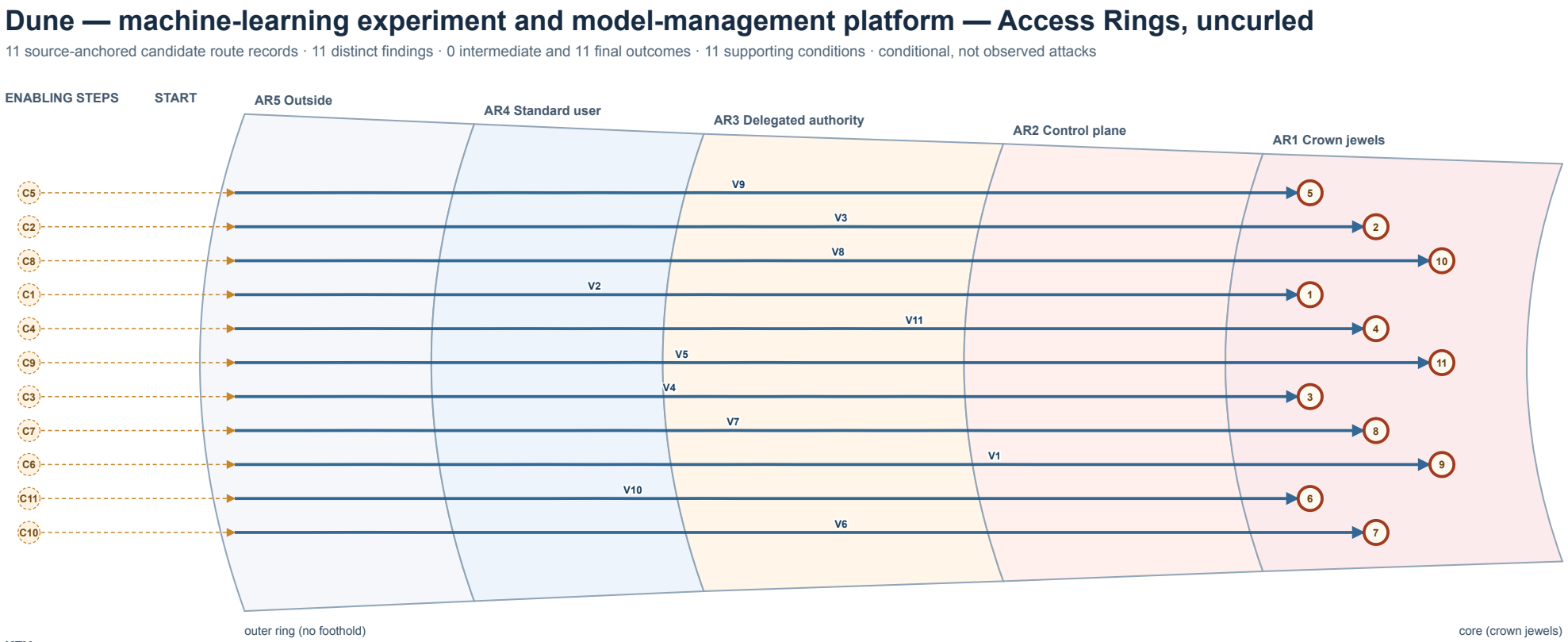
i1 · AR3 · administrator or staff session obtained
i2 · AR3 · administrator session obtained through an indirect route
i3 · AR4 · standard-user account or session obtained (a)
i4 · AR4 · standard-user account or session obtained (b)
i5 · AR4 · ordinary user's session reached by attacker content
i6 · AR5 · identifier or metadata disclosed

Enabling steps (not vulnerabilities)

C1 · non-default configuration or feature enabled (a)
C2 · victim opens attacker-influenced content
C3 · non-default configuration or feature enabled (b)

Rings are Access Rings (§3.4), drawn uncurled: outermost (AR5) at left, crown jewels (AR1) at right. Assignments follow the rule in §3.4; judgement calls are marked in the classification table. Exposure Ring unavailable (no deployment data).

Figure 5. Dune, machine-learning experiment and model-management platform: all 11 source-anchored route records.



KEY

- Code vulnerability (V#). Tail = ring held; head = ring reached; length = rings crossed.
- i1 — Intermediate outcome (i#): position or capability gained; the next vulnerability starts here.
- 1 — Final outcome, B1 Critical (#).
- 2 — Final outcome, B2 Major (#).
- Enabling step that puts an account, credential or a victim's session within reach: moves the attacker into that ring.
- C1 — Enabling step that gives no position (configuration, file, missing control): sets conditions for the next vulnerability.
- Dotted grey = start position not recorded in the ledger (arrow enters from the left of the rings).

Vulnerabilities (V# = code in the audit)

V1 · Dune-F01
V2 · Dune-F02
V3 · Dune-F03
V4 · Dune-F04
V5 · Dune-F05
V6 · Dune-F06
V7 · Dune-F07
V8 · Dune-F08
V9 · Dune-F09
V10 · Dune-F10
V11 · Dune-F11

Final outcomes (red = B1 Critical, amber = B2 Major)

1 · AR1 · B1 · server-side code execution (a)
2 · AR1 · B1 · server-side code execution (b)
3 · AR1 · B1 · server-side code execution (c)
4 · AR1 · B1 · server-side code execution (d)
5 · AR1 · B1 · server-side code execution (e)
6 · AR1 · B1 · server-side code execution (f)
7 · AR1 · B1 · server-side code execution (g)
8 · AR1 · B1 · server-side code execution (h)
9 · AR1 · B1 · server-side code execution (i)
10 · AR1 · B1 · server-side code execution (j)
11 · AR1 · B1 · server-side code execution (k)

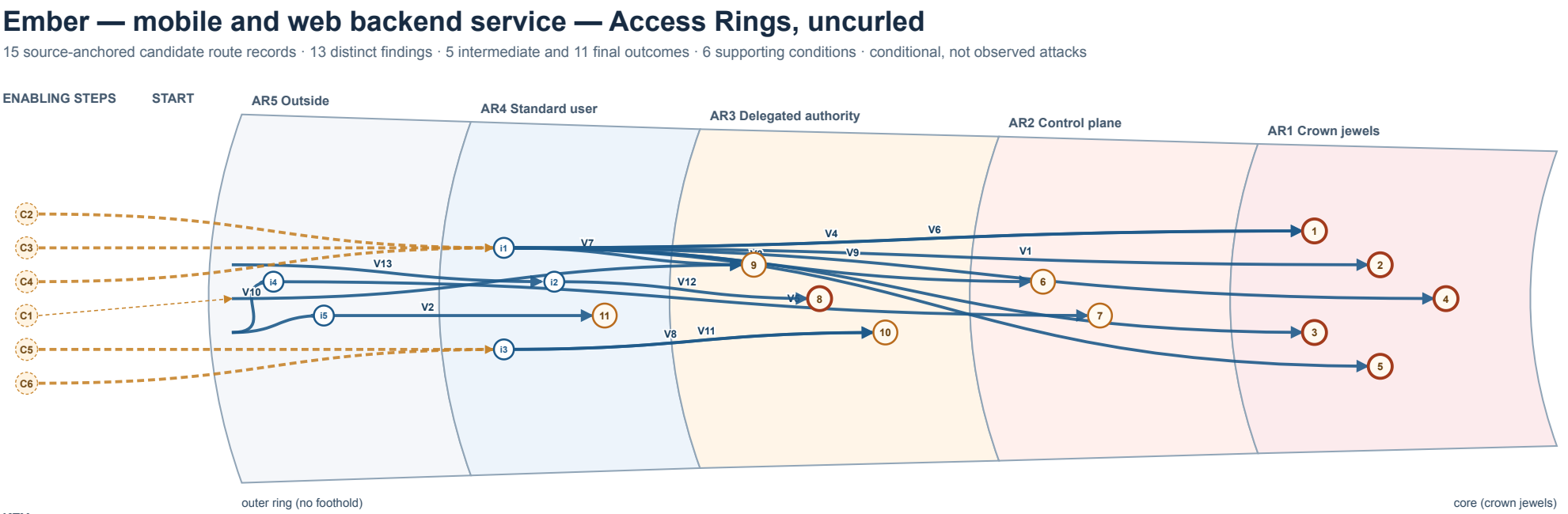
Intermediate outcomes (position or capability a finding gives; the next finding starts here)

Enabling steps (not vulnerabilities)

C1 · untrusted artifact selected for processing (a)
C2 · untrusted artifact selected for processing (b)
C3 · untrusted artifact selected for processing (c)
C4 · untrusted artifact selected for processing (d)
C5 · untrusted artifact selected for processing (e)
C6 · untrusted artifact selected for processing (f)
C7 · untrusted artifact selected for processing (g)
C8 · untrusted artifact selected for processing (h)
C9 · untrusted artifact selected for processing (i)
C10 · untrusted artifact selected for processing (j)
C11 · untrusted input reaches a command

Rings are Access Rings (§3.4), drawn uncurled: outermost (AR5) at left, crown jewels (AR1) at right. Assignments follow the rule in §3.4; judgement calls are marked in the classification table. Exposure Ring unavailable (no deployment data).

Figure 6. Ember, mobile and web backend service: all 15 source-anchored route records.



KEY

- Code vulnerability (V#). Tail = ring held; head = ring reached; length = rings crossed.
- (i1) Intermediate outcome (i#): position or capability gained; the next vulnerability starts here.
- (1) Final outcome, B1 Critical (#).
- (2) Final outcome, B2 Major (#).
- Enabling step that puts an account, credential or a victim's session within reach: moves the attacker into that ring.
- (C1) Enabling step that gives no position (configuration, file, missing control): sets conditions for the next vulnerability.
- Dotted grey = start position not recorded in the ledger (arrow enters from the left of the rings).

Vulnerabilities (V# = code in the audit)

V1 · Ember-F01
V2 · Ember-F02
V3 · Ember-F03
V4 · Ember-F04
V5 · Ember-F05
V6 · Ember-F06
V7 · Ember-F07
V8 · Ember-F08
V9 · Ember-F09
V10 · Ember-F10
V11 · Ember-F11
V12 · Ember-F12
V13 · Ember-F13

Final outcomes (red = B1 Critical, amber = B2 Major)

1 · AR1 · B1 · arbitrary database access
2 · AR1 · B1 · write access as any user (a)
3 · AR1 · B1 · write access as any user (b)
4 · AR1 · B1 · write access as any user (c)
5 · AR1 · B1 · write access as any user (d)
6 · AR2 · B2 · unauthorised modification or deletion
7 · AR2 · B2 · data disclosure beyond the caller's permissions
8 · AR3 · B1 · administrator or account session taken over (a)
9 · AR3 · B2 · secret or credential disclosure
10 · AR3 · B2 · administrator or account session taken over (c)
11 · AR4 · B2 · administrator or account session taken over (b)

Intermediate outcomes (position or capability a finding gives; the next finding starts here)

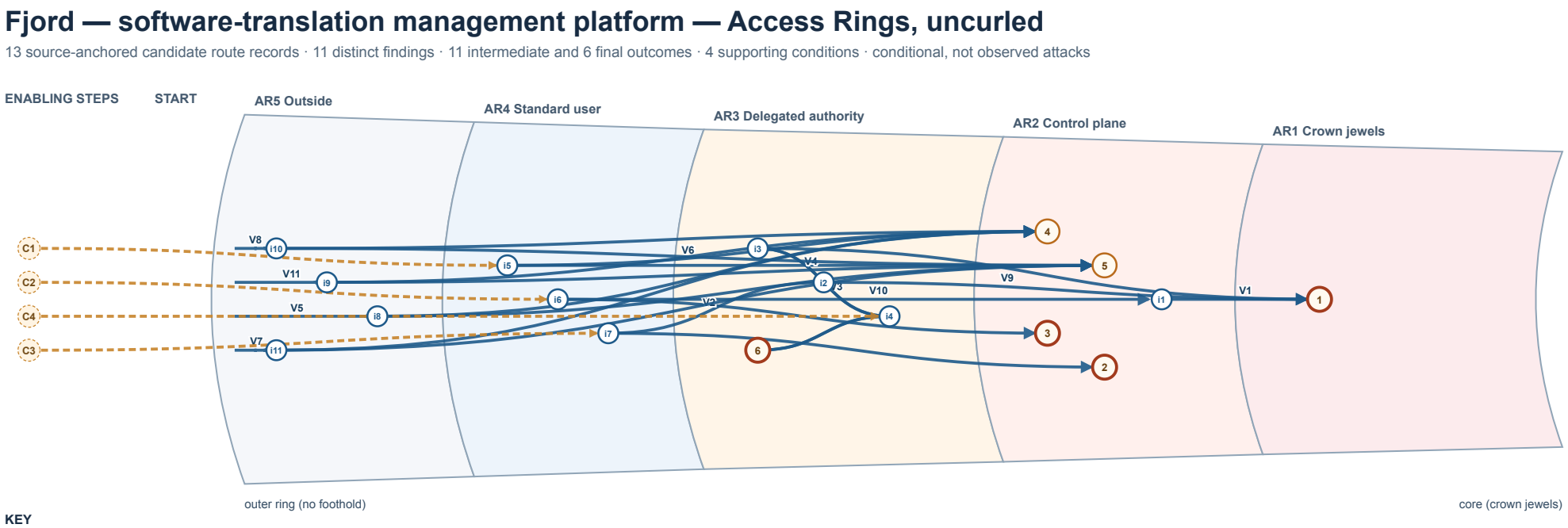
i1 · AR4 · standard-user account or credential held
i2 · AR4 · intermediate capability obtained
i3 · AR4 · ordinary user's session reached by attacker content
i4 · AR5 · identifier or metadata disclosed (a)
i5 · AR5 · identifier or metadata disclosed (b)

Enabling steps (not vulnerabilities)

C1 · missing rate or lockout control
C2 · credential, token or account obtained (phishing, issuance or reuse) (a)
C3 · credential, token or account obtained (phishing, issuance or reuse) (b)
C4 · credential, token or account obtained (phishing, issuance or reuse) (c)
C5 · victim opens attacker-influenced content (a)
C6 · victim opens attacker-influenced content (b)

Rings are Access Rings (§3.4), drawn uncurled: outermost (AR5) at left, crown jewels (AR1) at right. Assignments follow the rule in §3.4; judgement calls are marked in the classification table. Exposure Ring unavailable (no deployment data).

Figure 7. Fjord, software-translation management platform: all 13 source-anchored route records.



KEY

- Code vulnerability (V#). Tail = ring held; head = ring reached; length = rings crossed.
- i1 — Intermediate outcome (i#): position or capability gained; the next vulnerability starts here.
- 1 — Final outcome, B1 Critical (#).
- 2 — Final outcome, B2 Major (#).
- Enabling step that puts an account, credential or a victim's session within reach: moves the attacker into that ring.
- C1 — Enabling step that gives no position (configuration, file, missing control): sets conditions for the next vulnerability.
- Dotted grey = start position not recorded in the ledger (arrow enters from the left of the rings).

Vulnerabilities (V# = code in the audit)

V1 · Fjord-F01
V2 · Fjord-F02
V3 · Fjord-F03
V4 · Fjord-F04
V5 · Fjord-F05
V6 · Fjord-F06
V7 · Fjord-F07
V8 · Fjord-F08
V9 · Fjord-F09
V10 · Fjord-F10
V11 · Fjord-F11

Final outcomes (red = B1 Critical, amber = B2 Major)

1 · AR1 · B1 · server-side code execution (b)
2 · AR2 · B1 · platform-level administration taken over (a)
3 · AR2 · B1 · platform-level administration taken over (b)
4 · AR2 · B2 · server-side code execution (a)
5 · AR2 · B2 · trust or integrity compromise of a shared component
6 · AR3 · B1 · administrator or account session taken over

Intermediate outcomes (position or capability a finding gives; the next finding starts here)

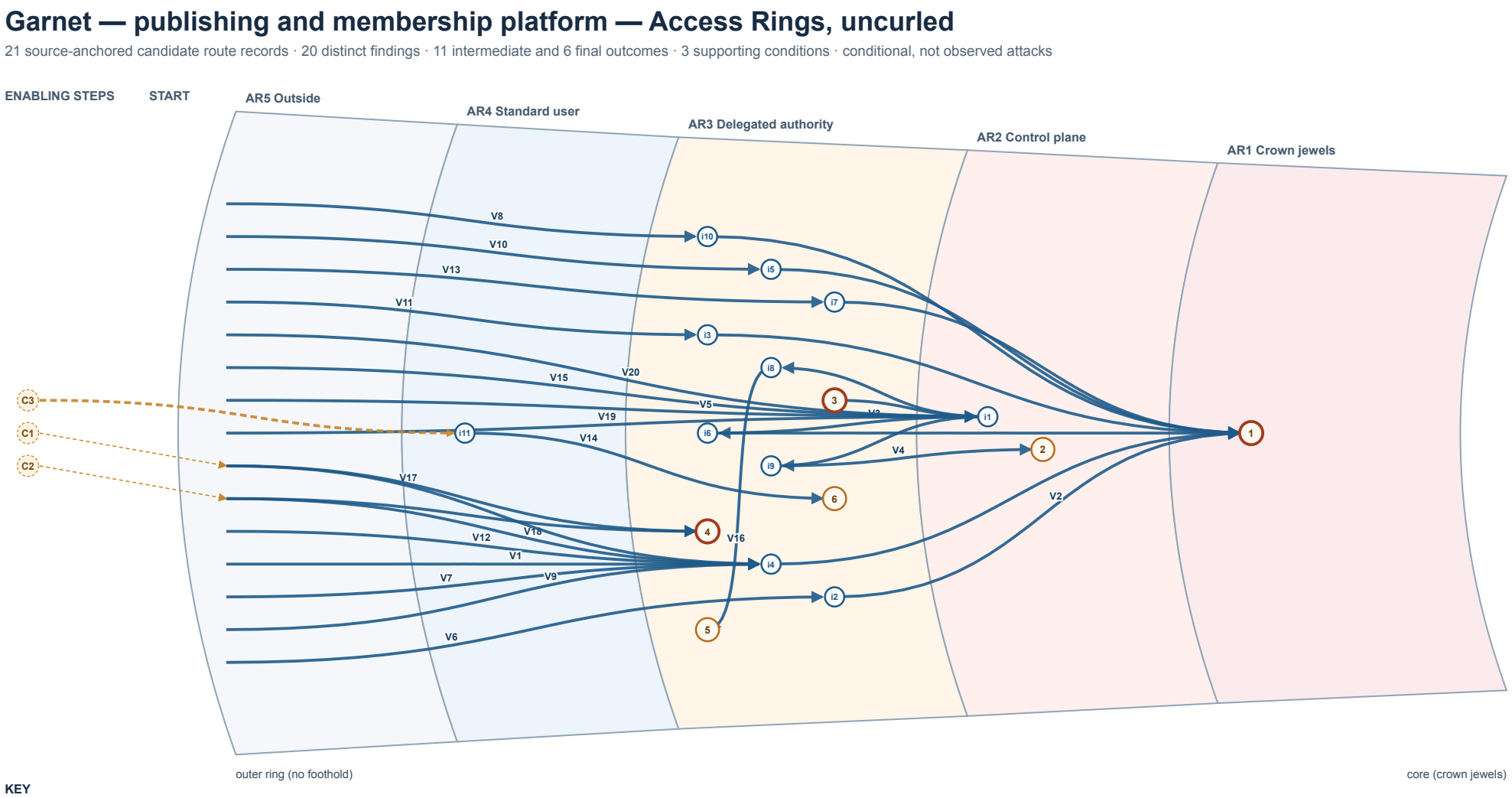
i1 · AR2 · privileged permission granted
i2 · AR3 · administrator or staff session obtained
i3 · AR3 · administrator session obtained through an indirect route
i4 · AR3 · victim's privileged session reached by attacker content
i5 · AR4 · standard-user account available (open registration or granted permission)
i6 · AR4 · standard-user account or credential held
i7 · AR4 · ordinary user's session reached by attacker content
i8 · AR5 · identifier or metadata disclosed (a)
i9 · AR5 · identifier or metadata disclosed (b)
i10 · AR5 · identifier or metadata disclosed (c)
i11 · AR5 · identifier or metadata disclosed (d)

Enabling steps (not vulnerabilities)

C1 · open registration or permissive default
C2 · credential, token or account obtained (phishing, issuance or reuse)
C3 · victim opens attacker-influenced content
C4 · victim opens attacker-influenced content (privileged user)

Rings are Access Rings (§3.4), drawn uncurled: outermost (AR5) at left, crown jewels (AR1) at right. Assignments follow the rule in §3.4; judgement calls are marked in the classification table. Exposure Ring unavailable (no deployment data).

Figure 8. Garnet, publishing and membership platform: all 21 source-anchored route records.



KEY

- Code vulnerability (V#). Tail = ring held; head = ring reached; length = rings crossed.
- i1 — Intermediate outcome (i#): position or capability gained; the next vulnerability starts here.
- 1 — Final outcome, B1 Critical (#).
- 2 — Final outcome, B2 Major (#).
- Enabling step that puts an account, credential or a victim's session within reach: moves the attacker into that ring.
- C1 — Enabling step that gives no position (configuration, file, missing control): sets conditions for the next vulnerability.
- Dotted grey = start position not recorded in the ledger (arrow enters from the left of the rings).

Vulnerabilities (V# = code in the audit)

V1 · Garnet-F01
V2 · Garnet-F02
V3 · Garnet-F03
V4 · Garnet-F04
V5 · Garnet-F05
V6 · Garnet-F06
V7 · Garnet-F07
V8 · Garnet-F08
V9 · Garnet-F09
V10 · Garnet-F10
V11 · Garnet-F11
V12 · Garnet-F12
V13 · Garnet-F13
V14 · Garnet-F14
V15 · Garnet-F15
V16 · Garnet-F16
V17 · Garnet-F17
V18 · Garnet-F18
V19 · Garnet-F19
V20 · Garnet-F20

Final outcomes (red = B1 Critical, amber = B2 Major)

1 · AR1 · B1 · server-side code execution
2 · AR2 · B2 · secret or credential disclosure
3 · AR3 · B1 · administrator or account session taken over (b)
4 · AR3 · B1 · administrator or account session taken over (c)
5 · AR3 · B2 · unauthorised modification or deletion
6 · AR3 · B2 · administrator or account session taken over (a)

Intermediate outcomes (position or capability a finding gives; the next finding starts here)

i1 · AR2 · secret or credential read
i2 · AR3 · administrator or staff session obtained (a)
i3 · AR3 · administrator session obtained through an indirect route (a)
i4 · AR3 · administrator session obtained through an indirect route (b)
i5 · AR3 · administrator session obtained through an indirect route (c)
i6 · AR3 · administrator session obtained through an indirect route (d)
i7 · AR3 · administrator session obtained through an indirect route (e)
i8 · AR3 · administrator or staff session obtained (b)
i9 · AR3 · administrator session obtained through an indirect route (f)
i10 · AR3 · administrator session obtained through an indirect route (g)
i11 · AR4 · standard-user account or credential held

Enabling steps (not vulnerabilities)

C1 · non-default configuration or feature enabled (a)
C2 · non-default configuration or feature enabled (b)
C3 · credential, token or account obtained (phishing, issuance or reuse)

Rings are Access Rings (§3.4), drawn uncurled: outermost (AR5) at left, crown jewels (AR1) at right. Assignments follow the rule in §3.4; judgement calls are marked in the classification table. Exposure Ring unavailable (no deployment data).

Figure 9. Harbor, workflow-orchestration platform: all 5 source-anchored route records.

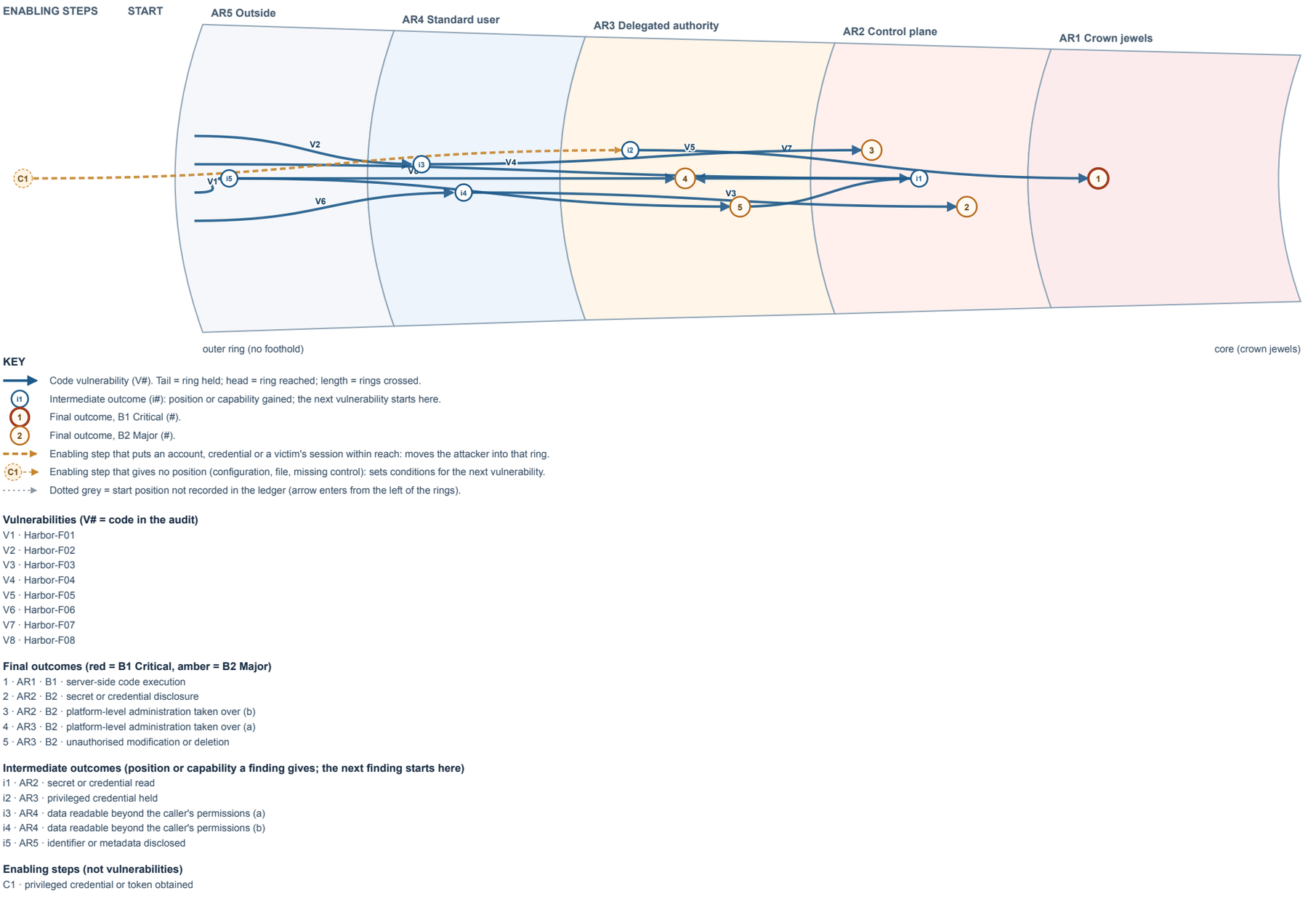
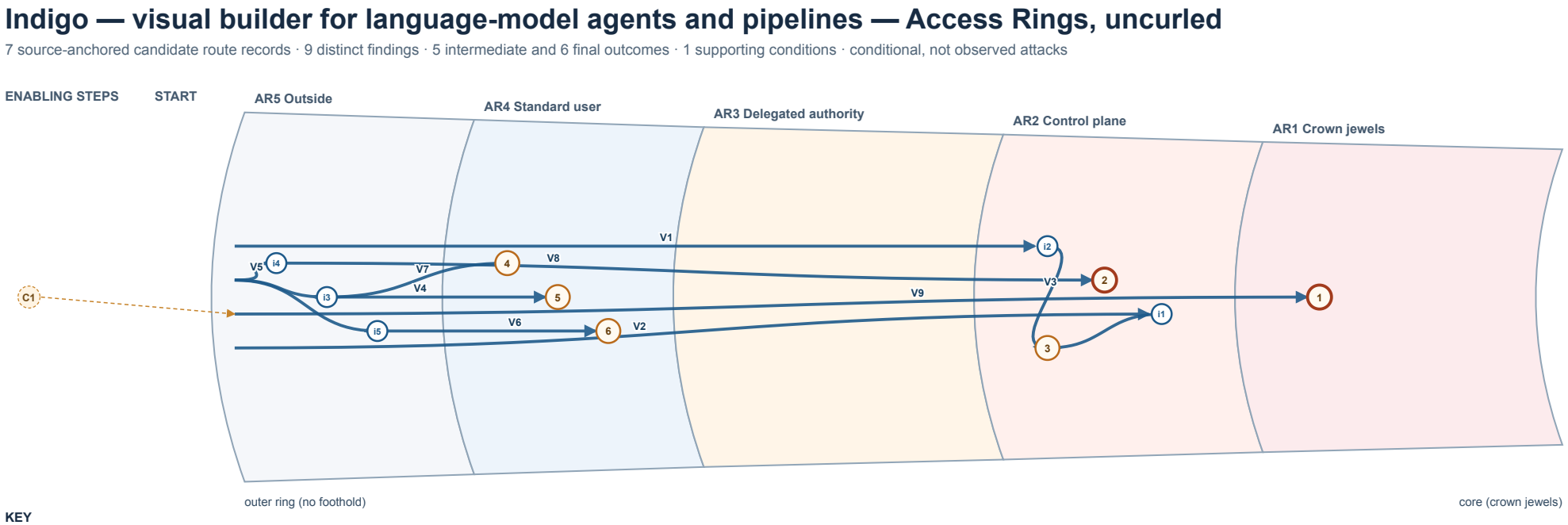


Figure 10. Indigo, visual builder for language-model agents and pipelines: all 7 source-anchored route records.



Vulnerabilities (V# = code in the audit)

V1 · Indigo-F01
V2 · Indigo-F02
V3 · Indigo-F03
V4 · Indigo-F04
V5 · Indigo-F05
V6 · Indigo-F06
V7 · Indigo-F07
V8 · Indigo-F08
V9 · Indigo-F09

Final outcomes (red = B1 Critical, amber = B2 Major)

1 · AR1 · B1 · server-side code execution
2 · AR2 · B1 · arbitrary file write on the server
3 · AR2 · B2 · secret or credential disclosure
4 · AR4 · B2 · data disclosure beyond the caller's permissions (a)
5 · AR4 · B2 · data disclosure beyond the caller's permissions (b)
6 · AR4 · B2 · unauthorised modification or deletion

Intermediate outcomes (position or capability a finding gives; the next finding starts here)

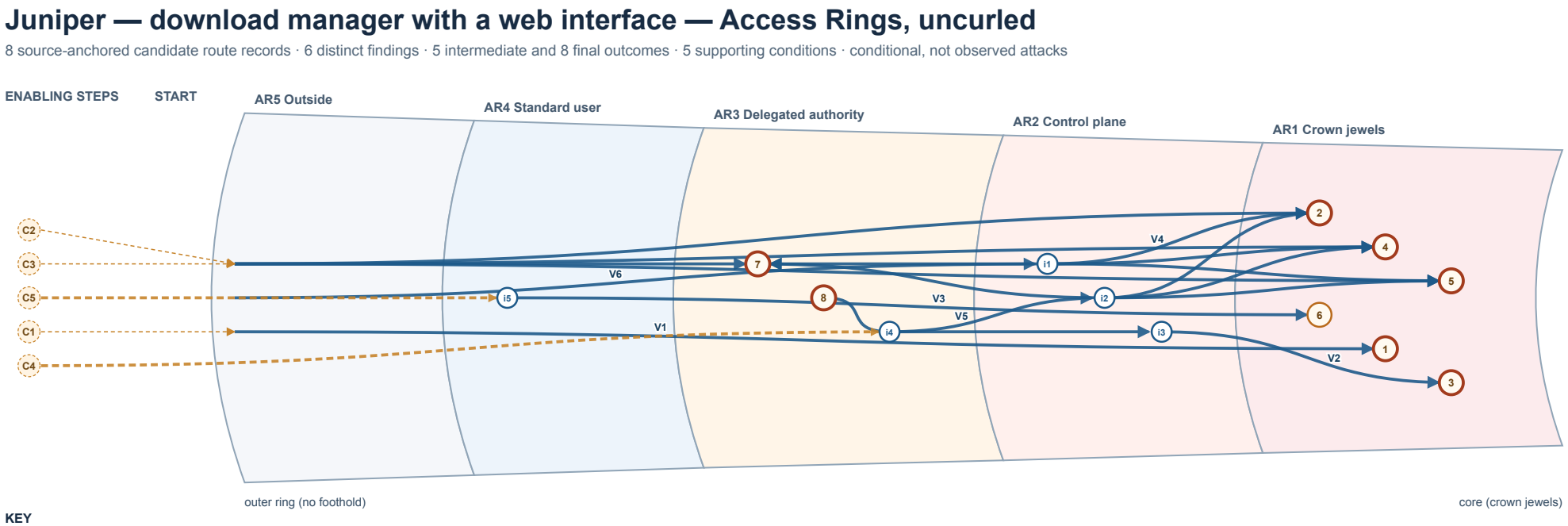
i1 · AR2 · secret or credential read (a)
i2 · AR2 · secret or credential read (b)
i3 · AR5 · identifier or metadata disclosed (a)
i4 · AR5 · identifier or metadata disclosed (b)
i5 · AR5 · identifier or metadata disclosed (c)

Enabling steps (not vulnerabilities)

C1 · publicly exposed endpoint or workflow

Rings are Access Rings (§3.4), drawn uncurled: outermost (AR5) at left, crown jewels (AR1) at right. Assignments follow the rule in §3.4; judgement calls are marked in the classification table. Exposure Ring unavailable (no deployment data).

Figure 11. Juniper, download manager with a web interface: all 8 source-anchored route records.



KEY

- Code vulnerability (V#). Tail = ring held; head = ring reached; length = rings crossed.
- (i1) Intermediate outcome (i#): position or capability gained; the next vulnerability starts here.
- (1) Final outcome, B1 Critical (#).
- (2) Final outcome, B2 Major (#).
- Enabling step that puts an account, credential or a victim's session within reach: moves the attacker into that ring.
- (C1) Enabling step that gives no position (configuration, file, missing control): sets conditions for the next vulnerability.
- Dotted grey = start position not recorded in the ledger (arrow enters from the left of the rings).

Vulnerabilities (V# = code in the audit)

V1 · Juniper-F01
V2 · Juniper-F02
V3 · Juniper-F03
V4 · Juniper-F04
V5 · Juniper-F05
V6 · Juniper-F06

Final outcomes (red = B1 Critical, amber = B2 Major)

1 · AR1 · B1 · server-side code execution (a)
2 · AR1 · B1 · server-side code execution (b)
3 · AR1 · B1 · server-side code execution (d)
4 · AR1 · B1 · server-side code execution (e)
5 · AR1 · B1 · server-side code execution (f)
6 · AR1 · B2 · arbitrary file write on the server
7 · AR3 · B1 · server-side code execution (c)
8 · AR3 · B1 · server-side code execution (g)

Intermediate outcomes (position or capability a finding gives; the next finding starts here)

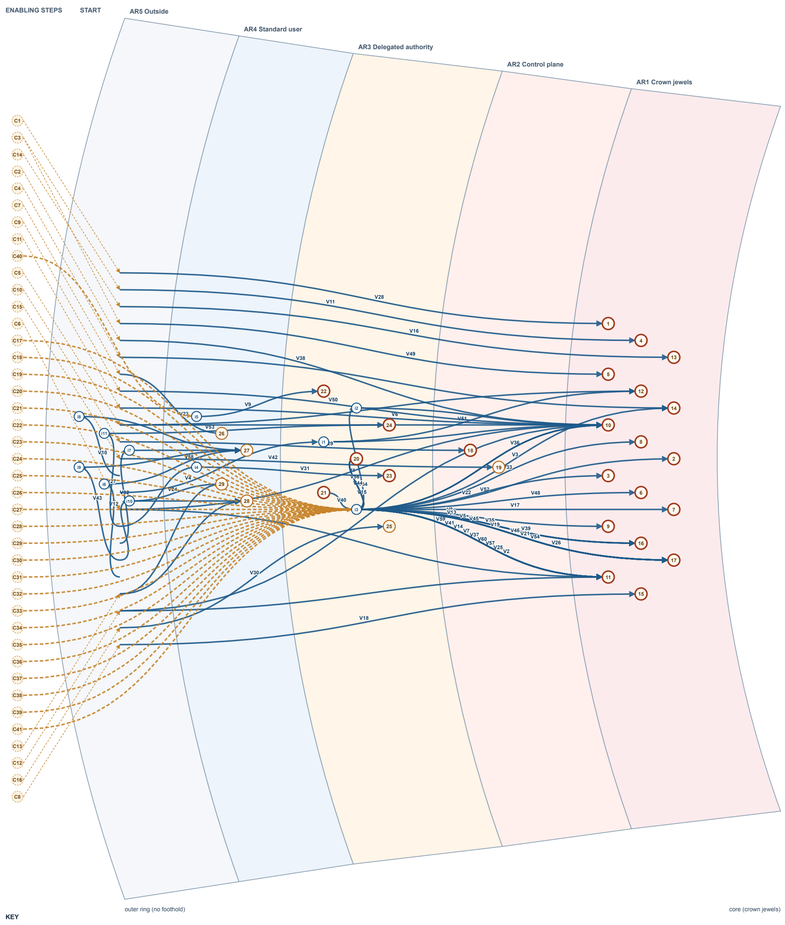
i1 · AR2 · secret or credential read
i2 · AR2 · platform-level administration obtained (a)
i3 · AR2 · platform-level administration obtained (b)
i4 · AR3 · victim's privileged session reached by attacker content
i5 · AR4 · ordinary user's session reached by attacker content

Enabling steps (not vulnerabilities)

C1 · non-default configuration or feature enabled (a)
C2 · non-default configuration or feature enabled (b)
C3 · missing rate or lockout control
C4 · victim opens attacker-influenced content (privileged user)
C5 · victim opens attacker-influenced content

Rings are Access Rings (§3.4), drawn uncurled: outermost (AR5) at left, crown jewels (AR1) at right. Assignments follow the rule in §3.4; judgement calls are marked in the classification table. Exposure Ring unavailable (no deployment data).

Figure 12. Kestrel, enterprise wiki and application platform: all 63 source-anchored route records.



KEY

- Code vulnerability (V#). Tail = ring held; head = ring reached; length = rings crossed.
- (i#) Intermediate outcome (i#): position or capability gained; the next vulnerability starts here.
- 1 Final outcome, B1 Critical (#).
- 2 Final outcome, B2 Major (#).
- Enabling step that puts an account, credential or a victim's session within reach: moves the attacker into that ring.
- (C#) Enabling step that gives no position (configuration, file, missing control); sets conditions for the next vulnerability.
- Dotted grey + start position not recorded in the ledger (arrow enters from the left of the rings).

Vulnerabilities (V# = code in the audit)

V1 · Kestrel-F01
V2 · Kestrel-F02
V3 · Kestrel-F03
V4 · Kestrel-F04
V5 · Kestrel-F05
V6 · Kestrel-F06
V7 · Kestrel-F07
V8 · Kestrel-F08
V9 · Kestrel-F09
V10 · Kestrel-F10
V11 · Kestrel-F11
V12 · Kestrel-F12
V13 · Kestrel-F13
V14 · Kestrel-F14
V15 · Kestrel-F15
V16 · Kestrel-F16
V17 · Kestrel-F17
V18 · Kestrel-F18
V19 · Kestrel-F19
V20 · Kestrel-F20
V21 · Kestrel-F21
V22 · Kestrel-F22
V23 · Kestrel-F23
V24 · Kestrel-F24
V25 · Kestrel-F25
V26 · Kestrel-F26
V27 · Kestrel-F27
V28 · Kestrel-F28
V29 · Kestrel-F29
V30 · Kestrel-F30
V31 · Kestrel-F31
V32 · Kestrel-F32
V33 · Kestrel-F33
V34 · Kestrel-F34
V35 · Kestrel-F35
V36 · Kestrel-F36
V37 · Kestrel-F37
V38 · Kestrel-F38
V39 · Kestrel-F39
V40 · Kestrel-F40
V41 · Kestrel-F41
V42 · Kestrel-F42
V43 · Kestrel-F43
V44 · Kestrel-F44
V45 · Kestrel-F45
V46 · Kestrel-F46
V47 · Kestrel-F47
V48 · Kestrel-F48
V49 · Kestrel-F49
V50 · Kestrel-F50
V51 · Kestrel-F51
V52 · Kestrel-F52
V53 · Kestrel-F53
V54 · Kestrel-F54
V55 · Kestrel-F55
V56 · Kestrel-F56
V57 · Kestrel-F57
V58 · Kestrel-F58
V59 · Kestrel-F59
V60 · Kestrel-F60

Final outcomes (red = B1 Critical, amber = B2 Major)

1 · AR1 · B1 · server-side code execution (a)
2 · AR1 · B1 · server-side code execution (b)
3 · AR1 · B1 · server-side code execution (c)
4 · AR1 · B1 · arbitrary database access (a)
5 · AR1 · B1 · arbitrary database access (b)
6 · AR1 · B1 · server-side code execution (d)
7 · AR1 · B1 · server-side code execution (e)
8 · AR1 · B1 · server-side code execution (f)
9 · AR1 · B1 · server-side code execution (g)
10 · AR1 · B1 · server-side code execution (h)
11 · AR1 · B1 · server-side code execution (i)
12 · AR1 · B1 · server-side code execution (j)
13 · AR1 · B1 · server-side code execution (k)
14 · AR1 · B1 · server-side code execution (l)
15 · AR1 · B1 · arbitrary file write on the server
16 · AR1 · B1 · server-side code execution (o)
17 · AR1 · B1 · server-side code execution (p)
18 · AR2 · B1 · secret or credential disclosure (b)
19 · AR2 · B2 · secret or credential disclosure (a)
20 · AR3 · B1 · administrator or account session taken over (a)
21 · AR3 · B1 · server-side code execution (m)
22 · AR3 · B1 · server-side code execution (n)
23 · AR3 · B1 · administrator or account session taken over (c)
24 · AR3 · B1 · administrator or account session taken over (d)
25 · AR3 · B2 · administrator or account session taken over (b)
26 · AR4 · B2 · data disclosure beyond the caller's permissions (a)
27 · AR4 · B2 · data disclosure beyond the caller's permissions (b)
28 · AR4 · B2 · data disclosure beyond the caller's permissions (c)
29 · AR4 · B2 · data disclosure beyond the caller's permissions (d)

Intermediate outcomes (position or capability a finding gives; the next finding starts here)

i1 · AR3 · administrator or staff session obtained
i2 · AR3 · administrator session obtained through an indirect route
i3 · AR3 · victim's privileged session reached by attacker content
i4 · AR4 · stored credential readable
i5 · AR4 · ordinary user's session reached by attacker content
i6 · AR5 · identifier or metadata disclosed (a)
i7 · AR5 · identifier or metadata disclosed (b)
i8 · AR5 · identifier or metadata disclosed (c)
i9 · AR5 · identifier or metadata disclosed (d)
i10 · AR5 · identifier or metadata disclosed (e)
i11 · AR5 · identifier or metadata disclosed (f)

Enabling steps (not vulnerabilities)

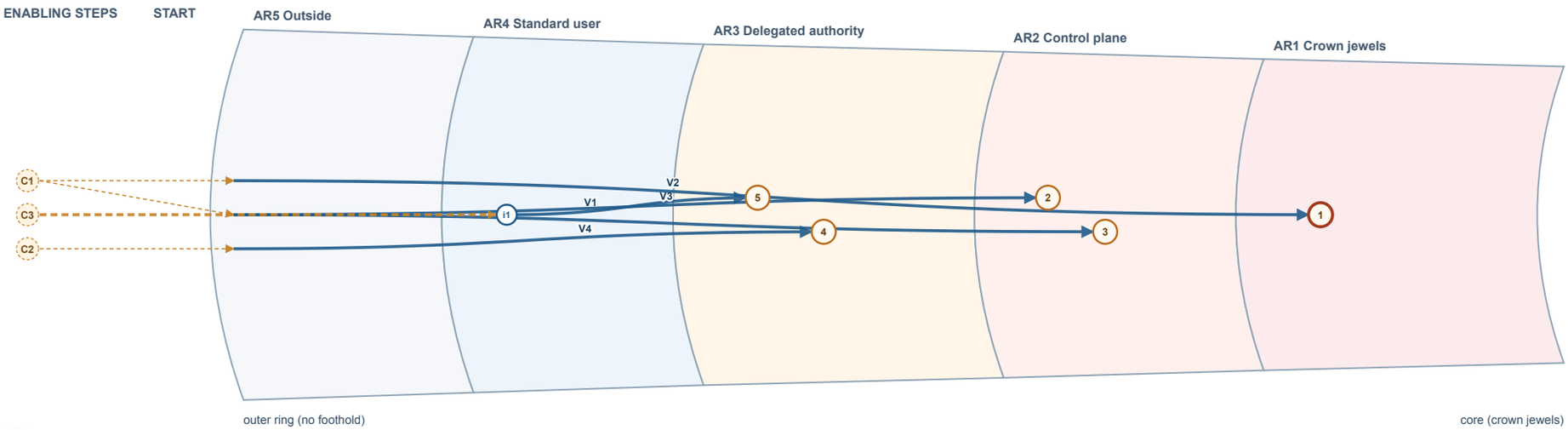
C1 · non-default configuration or feature enabled (a)
C2 · non-default configuration or feature enabled (b)
C3 · non-default configuration or feature enabled (c)
C4 · non-default configuration or feature enabled (d)
C5 · non-default configuration or feature enabled (e)
C6 · non-default configuration or feature enabled (f)
C7 · non-default configuration or feature enabled (g)
C8 · non-default configuration or feature enabled (h)
C9 · non-default configuration or feature enabled (i)
C10 · non-default configuration or feature enabled (j)
C11 · non-default configuration or feature enabled (k)
C12 · non-default configuration or feature enabled (l)
C13 · non-default configuration or feature enabled (m)
C14 · non-default configuration or feature enabled (n)
C15 · non-default configuration or feature enabled (o)
C16 · non-default configuration or feature enabled (p)
C17 · victim opens attacker-influenced content (privileged user) (a)
C18 · victim opens attacker-influenced content (privileged user) (b)
C19 · victim opens attacker-influenced content (privileged user) (c)
C20 · victim opens attacker-influenced content (privileged user) (d)
C21 · victim opens attacker-influenced content (privileged user) (e)
C22 · victim opens attacker-influenced content (privileged user) (f)
C23 · victim opens attacker-influenced content (privileged user) (g)
C24 · victim opens attacker-influenced content (privileged user) (h)
C25 · victim opens attacker-influenced content (privileged user) (i)
C26 · victim opens attacker-influenced content (privileged user) (j)
C27 · victim opens attacker-influenced content (privileged user) (k)
C28 · victim opens attacker-influenced content (privileged user) (l)
C29 · victim opens attacker-influenced content (privileged user) (m)
C30 · victim opens attacker-influenced content (privileged user) (n)
C31 · victim opens attacker-influenced content (privileged user) (o)
C32 · victim opens attacker-influenced content (privileged user) (p)
C33 · victim opens attacker-influenced content (privileged user) (q)
C34 · victim opens attacker-influenced content (privileged user) (r)
C35 · victim opens attacker-influenced content (privileged user) (s)
C36 · victim opens attacker-influenced content (privileged user) (t)
C37 · victim opens attacker-influenced content (privileged user) (u)
C38 · victim opens attacker-influenced content (privileged user) (v)
C39 · victim opens attacker-influenced content (privileged user) (w)
C40 · victim opens attacker-influenced content
C41 · victim opens attacker-influenced content (privileged user) (x)

Rings are Access Rings (§3.4), drawn uncurled: outermost (AR5) at left, crown jewels (AR1) at right. Assignments follow the rule in §3.4; judgement calls are marked in the classification table. Exposure Ring unavailable (no deployment data).

Figure 13. Lumen, web-interface toolkit for machine-learning demos: all 5 source-anchored route records.

Lumen — web-interface toolkit for machine-learning demos — Access Rings, uncurled

5 source-anchored candidate route records · 4 distinct findings · 1 intermediate and 5 final outcomes · 3 supporting conditions · conditional, not observed attacks



KEY

- Code vulnerability (V#). Tail = ring held; head = ring reached; length = rings crossed.
- i1 — Intermediate outcome (i#): position or capability gained; the next vulnerability starts here.
- 1 — Final outcome, B1 Critical (#).
- 2 — Final outcome, B2 Major (#).
- Enabling step that puts an account, credential or a victim's session within reach: moves the attacker into that ring.
- C1 — Enabling step that gives no position (configuration, file, missing control): sets conditions for the next vulnerability.
- Dotted grey = start position not recorded in the ledger (arrow enters from the left of the rings).

Vulnerabilities (V# = code in the audit)

V1 · Lumen-F01
V2 · Lumen-F02
V3 · Lumen-F03
V4 · Lumen-F04

Final outcomes (red = B1 Critical, amber = B2 Major)

1 · AR1 · B1 · server-side code execution
2 · AR2 · B2 · secret or credential disclosure (a)
3 · AR2 · B2 · trust or integrity compromise of a shared component
4 · AR3 · B2 · secret or credential disclosure (b)
5 · AR3 · B2 · secret or credential disclosure (c)

Intermediate outcomes (position or capability a finding gives; the next finding starts here)

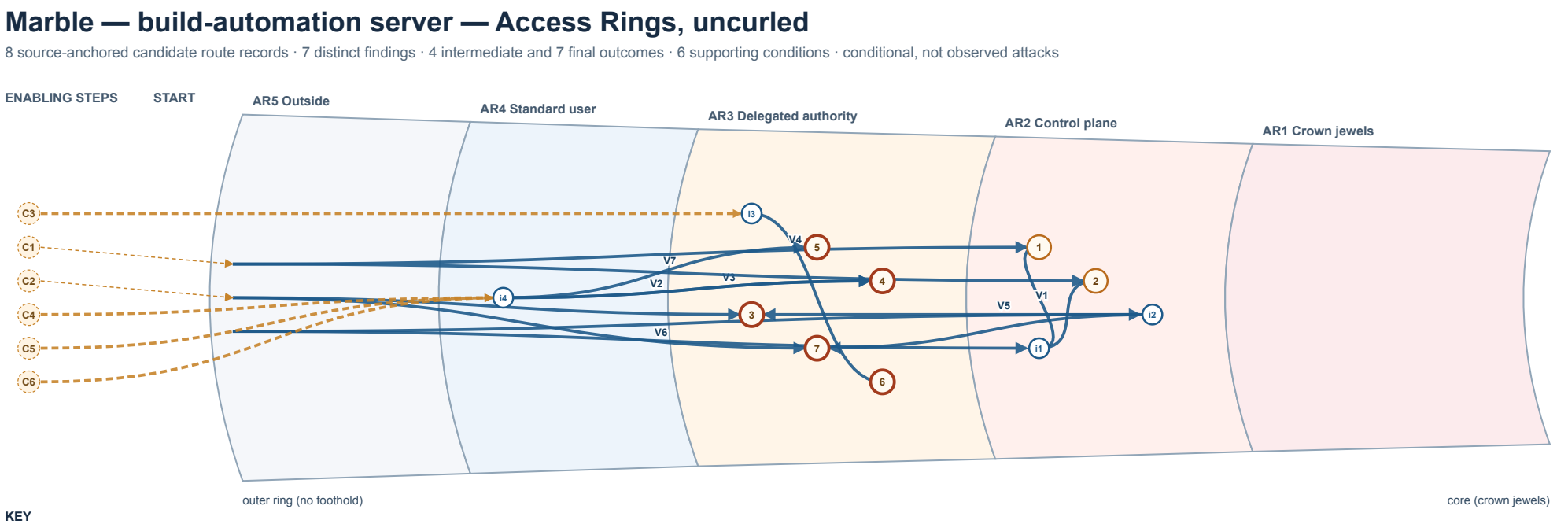
i1 · AR4 · ordinary user's session reached by attacker content

Enabling steps (not vulnerabilities)

C1 · non-default configuration or feature enabled
C2 · missing rate or lockout control
C3 · victim opens attacker-influenced content

Rings are Access Rings (§3.4), drawn uncurled: outermost (AR5) at left, crown jewels (AR1) at right. Assignments follow the rule in §3.4; judgement calls are marked in the classification table. Exposure Ring unavailable (no deployment data).

Figure 14. Marble, build-automation server: all 8 source-anchored route records.



KEY

- Code vulnerability (V#). Tail = ring held; head = ring reached; length = rings crossed.
- (i1) Intermediate outcome (i#): position or capability gained; the next vulnerability starts here.
- (1) Final outcome, B1 Critical (#).
- (2) Final outcome, B2 Major (#).
- Enabling step that puts an account, credential or a victim's session within reach: moves the attacker into that ring.
- (C1) Enabling step that gives no position (configuration, file, missing control): sets conditions for the next vulnerability.
- Dotted grey = start position not recorded in the ledger (arrow enters from the left of the rings).

Vulnerabilities (V# = code in the audit)

V1 · Marble-F01
V2 · Marble-F02
V3 · Marble-F03
V4 · Marble-F04
V5 · Marble-F05
V6 · Marble-F06
V7 · Marble-F07

Final outcomes (red = B1 Critical, amber = B2 Major)

1 · AR2 · B2 · secret or credential disclosure (a)
2 · AR2 · B2 · secret or credential disclosure (b)
3 · AR3 · B1 · administrator or account session taken over (a)
4 · AR3 · B1 · administrator or account session taken over (b)
5 · AR3 · B1 · administrator or account session taken over (c)
6 · AR3 · B1 · administrator or account session taken over (d)
7 · AR3 · B1 · administrator or account session taken over (e)

Intermediate outcomes (position or capability a finding gives; the next finding starts here)

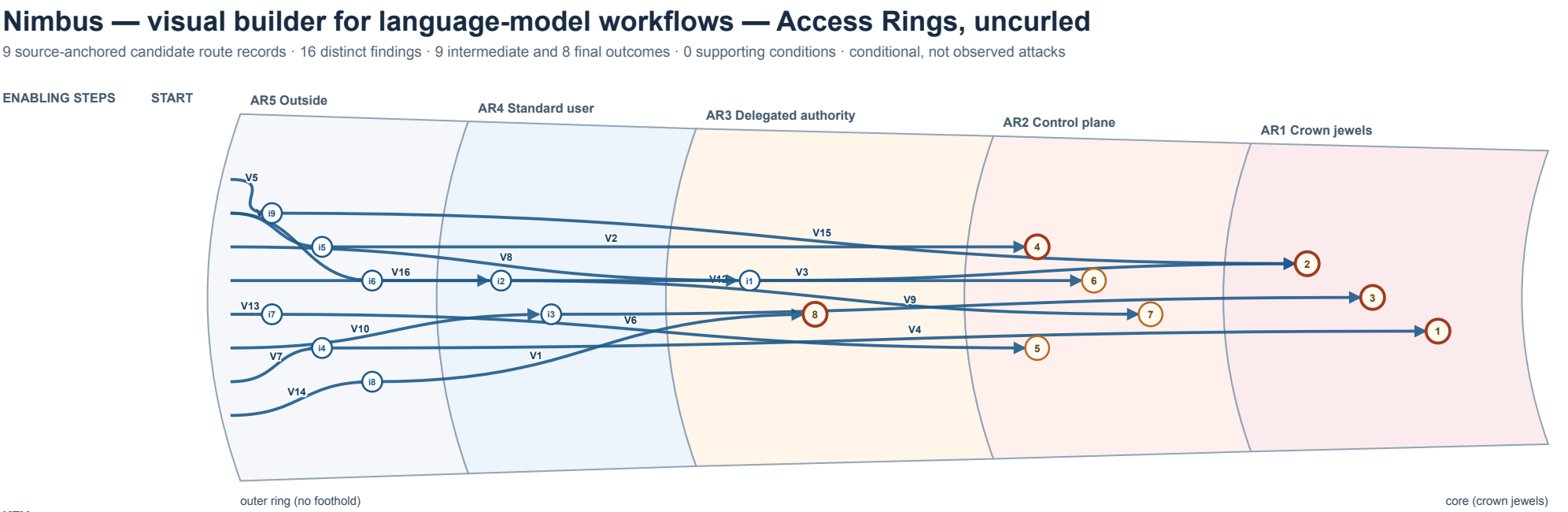
i1 · AR2 · secret or credential read (a)
i2 · AR2 · secret or credential read (b)
i3 · AR3 · victim's privileged session reached by attacker content
i4 · AR4 · ordinary user's session reached by attacker content

Enabling steps (not vulnerabilities)

C1 · non-default configuration or feature enabled (a)
C2 · non-default configuration or feature enabled (b)
C3 · victim opens attacker-influenced content (privileged user)
C4 · victim opens attacker-influenced content (a)
C5 · victim opens attacker-influenced content (b)
C6 · victim opens attacker-influenced content (c)

Rings are Access Rings (§3.4), drawn uncurled: outermost (AR5) at left, crown jewels (AR1) at right. Assignments follow the rule in §3.4; judgement calls are marked in the classification table. Exposure Ring unavailable (no deployment data).

Figure 15. Nimbus, visual builder for language-model workflows: all 9 source-anchored route records.



KEY

- Code vulnerability (V#). Tail = ring held; head = ring reached; length = rings crossed.
- (i1) Intermediate outcome (i#): position or capability gained; the next vulnerability starts here.
- (1) Final outcome, B1 Critical (#).
- (2) Final outcome, B2 Major (#).
- Enabling step that puts an account, credential or a victim's session within reach: moves the attacker into that ring.
- (C1) Enabling step that gives no position (configuration, file, missing control): sets conditions for the next vulnerability.
- Dotted grey = start position not recorded in the ledger (arrow enters from the left of the rings).

Vulnerabilities (V# = code in the audit)

V1 · Nimbus-F01
V2 · Nimbus-F02
V3 · Nimbus-F03
V4 · Nimbus-F04
V5 · Nimbus-F05
V6 · Nimbus-F06
V7 · Nimbus-F07
V8 · Nimbus-F08
V9 · Nimbus-F09
V10 · Nimbus-F10
V11 · Nimbus-F11
V12 · Nimbus-F12
V13 · Nimbus-F13
V14 · Nimbus-F14
V15 · Nimbus-F15
V16 · Nimbus-F16

Final outcomes (red = B1 Critical, amber = B2 Major)

1 · AR1 · B1 · server-side code execution (a)
2 · AR1 · B1 · server-side code execution (b)
3 · AR1 · B1 · server-side code execution (c)
4 · AR2 · B1 · secret or credential disclosure (d)
5 · AR2 · B2 · secret or credential disclosure (a)
6 · AR2 · B2 · secret or credential disclosure (b)
7 · AR2 · B2 · secret or credential disclosure (c)
8 · AR3 · B1 · administrator or account session taken over

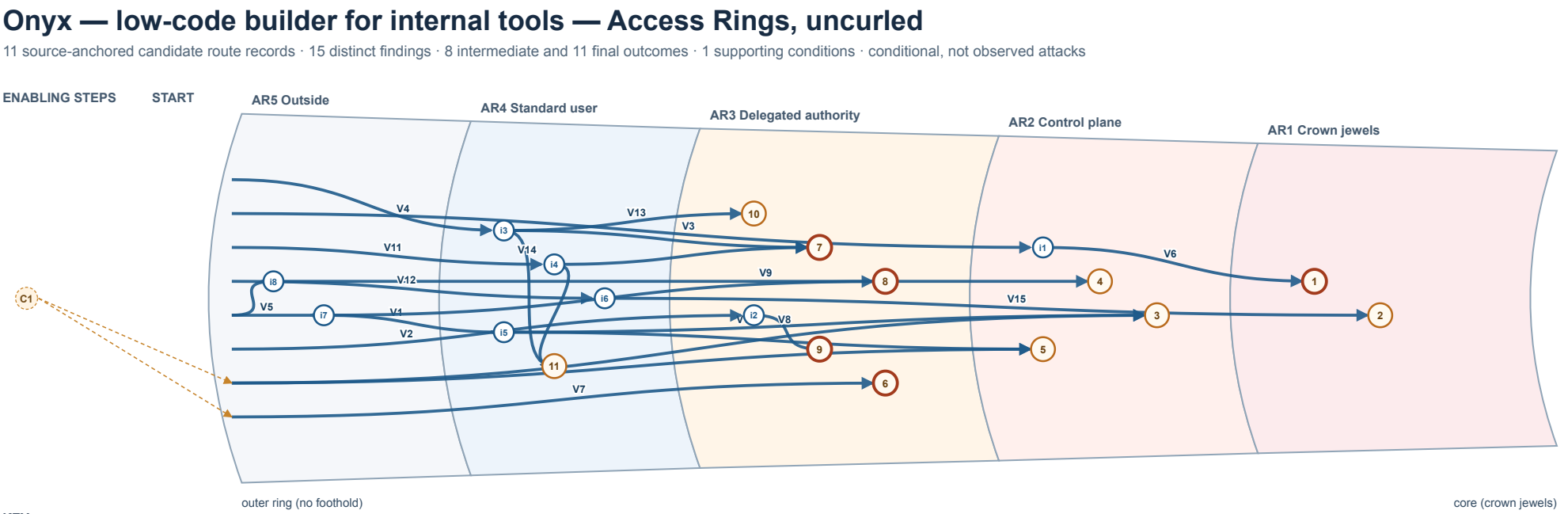
Intermediate outcomes (position or capability a finding gives; the next finding starts here)

i1 · AR3 · administrator session obtained through an indirect route
i2 · AR4 · standard-user account or session obtained
i3 · AR4 · attacker-controlled data written
i4 · AR5 · identifier or metadata disclosed (a)
i5 · AR5 · identifier or metadata disclosed (b)
i6 · AR5 · identifier or metadata disclosed (c)
i7 · AR5 · identifier or metadata disclosed (d)
i8 · AR5 · identifier or metadata disclosed (e)
i9 · AR5 · identifier or metadata disclosed (f)

Enabling steps (not vulnerabilities)

Rings are Access Rings (§3.4), drawn uncurled: outermost (AR5) at left, crown jewels (AR1) at right. Assignments follow the rule in §3.4; judgement calls are marked in the classification table. Exposure Ring unavailable (no deployment data).

Figure 16. Onyx, low-code builder for internal tools: all 11 source-anchored route records.



KEY

- Code vulnerability (V#). Tail = ring held; head = ring reached; length = rings crossed.
- (i1) Intermediate outcome (i#): position or capability gained; the next vulnerability starts here.
- (1) Final outcome, B1 Critical (#).
- (2) Final outcome, B2 Major (#).
- Enabling step that puts an account, credential or a victim's session within reach: moves the attacker into that ring.
- (C1) Enabling step that gives no position (configuration, file, missing control): sets conditions for the next vulnerability.
- Dotted grey = start position not recorded in the ledger (arrow enters from the left of the rings).

Vulnerabilities (V# = code in the audit)

V1 · Onyx-F01
V2 · Onyx-F02
V3 · Onyx-F03
V4 · Onyx-F04
V5 · Onyx-F05
V6 · Onyx-F06
V7 · Onyx-F07
V8 · Onyx-F08
V9 · Onyx-F09
V10 · Onyx-F10
V11 · Onyx-F11
V12 · Onyx-F12
V13 · Onyx-F13
V14 · Onyx-F14
V15 · Onyx-F15

Final outcomes (red = B1 Critical, amber = B2 Major)

1 · AR1 · B1 · data disclosure beyond the caller's permissions (a)
2 · AR1 · B2 · arbitrary database access
3 · AR2 · B2 · trust or integrity compromise of a shared component (a)
4 · AR2 · B2 · data disclosure beyond the caller's permissions (b)
5 · AR2 · B2 · trust or integrity compromise of a shared component (b)
6 · AR3 · B1 · administrator or account session taken over (b)
7 · AR3 · B1 · administrator or account session taken over (c)
8 · AR3 · B1 · administrator or account session taken over (d)
9 · AR3 · B1 · protected outcome reached (b)
10 · AR3 · B2 · protected outcome reached (a)
11 · AR4 · B2 · administrator or account session taken over (a)

Intermediate outcomes (position or capability a finding gives; the next finding starts here)

i1 · AR2 · secret or credential read
i2 · AR3 · administrator or staff session obtained
i3 · AR4 · session cookie readable by script
i4 · AR4 · cross-site request protection bypassed
i5 · AR4 · standard-user account or session obtained
i6 · AR4 · attacker-controlled data written
i7 · AR5 · identifier or metadata disclosed (a)
i8 · AR5 · identifier or metadata disclosed (b)

Enabling steps (not vulnerabilities)

C1 · non-default configuration or feature enabled

Rings are Access Rings (§3.4), drawn uncurled: outermost (AR5) at left, crown jewels (AR1) at right. Assignments follow the rule in §3.4; judgement calls are marked in the classification table. Exposure Ring unavailable (no deployment data).

Figures 2–16. Shared vulnerability arrows appear once per repository graph. Routes remain conditional candidates as described in §4.3.

6. Discussion

6.1 What selective hardening changes—and what it does not

Securing an exposed direct route changes what an attacker can reach most easily, but leaves the rest of the application's code to assess. The six changed forks retain route records through other weaknesses and ordinary conditions, some labelled B1 Critical. Lumen also shows that one fix can remove several proposed chains sharing a vulnerable step. The relevant result is therefore the path left after each named fix: which handoffs still connect to which protected outcome, and under what conditions?

The 1,000-to-ten example illustrates why a small backlog does not establish low risk; it is not a reduction measured here. Our intervention targeted ten finding IDs across five forks and an administrative credential condition in a sixth. It tests how named fixes affect recorded routes, rather than how often such chains occur across production software.

6.2 Implications for remediation and scanners

Remediation should follow outcomes and routes. A flaw with a bounded effect alone may become urgent if it supplies the identifier or authority needed by a critical path. A proposal labelled B1 Critical with an impossible handoff should be rejected. Teams can then see whether one fix interrupts several routes or whether independent routes to the same outcome need separate controls.

Two scanners with equal finding recall can leave different protected outcomes reachable through their misses. RoR's benchmark residual-risk measure makes that difference visible. The six modified OSS forks address a related question: which routes remain after documented fixes? Both views move evaluation from finding totals toward consequences.

6.3 Agent-assisted search

Agent-assisted search can explore cross-file and cross-feature paths that single-finding triage misses. Its value here is to propose consequential handoffs for source review, not to assume every proposed path works. Defence in depth means tracing and breaking those paths in code so that one failed control does not grant the next protected capability.

6.4 From a list of findings to a map of chokepoints

A list of vulnerabilities ranks each finding alone. It cannot show which findings hand an attacker the same position, or which single finding stands between that position and the crown jewels. The Access Ring maps add both. Each circle is a position that several findings can reach, and each arrow is a finding that moves the attacker from one position to the next. Counting the routes that share a circle or a finding turns the same inventory into a statement about where a fix removes the most paths. Three patterns appear across the fifteen maps.

Dispersed chains. Harbor and Onyx are ordinary applications whose routes do not share a centre. No circle carries more than two route records (Harbor: five records over eight findings, each circle on one record; Onyx: eleven records over fifteen findings, the busiest circle on two). Reaching AR1 takes a separate

chain each time: one route in Harbor and two, through different findings, in Onyx. Here the map confirms that no single fix removes the exposure, so findings are best treated one by one, and the ring view shows which of them completes a route into AR1.

A convergent chokepoint. Garnet has twenty-one route records, and twelve of them end in AR1 through one finding, Garnet-F02 (server-side code execution). Twelve different findings lead into that finding through seven circles at AR3, all of which are administrator or owner positions. The busiest, i4 (an administrator session reached through a victim), is entered by six findings alone. A list presents these thirteen findings as separate items. The map shows that one of them guards the route to AR1 for the other twelve, so correcting the step between the AR3 circles and AR1 removes every AR1 route in the ledger, while correcting any one entry finding removes only its own route. A second, smaller chokepoint sits one ring out: Garnet-F03 carries the four routes that start from an offline-recovered staff credential.

Interlocking chains. Basalt has forty-nine route records over twenty-two findings and twenty intermediate circles, and its chains cross. Three structures stand out. First, one finding, Basalt-F14, ends all thirteen AR1 routes. Second, the circle for a delegated administrator session (i13) carries twenty of the forty-nine records, entered by ten findings and left through two. Third, the standard-user circle (i17) is entered only through supporting steps (seven credentials, registration tokens and issued accounts), never through a code flaw, and eight findings leave it towards seven different outcomes. Because the routes leaving i17 end at AR2 or AR3, reaching AR1 from there depends on joining them with the platform-administrator routes, which the composed-path register treats as candidate joins still subject to token and role review. The practical reading is that the controls behind those seven steps (how initial access tokens are issued, who may register, how credentials are handled) deserve review to the same standard as the code, since a failure in any one of them opens several routes at once.

The maps therefore support three working rules: prioritise the finding that every route to a ring shares; review the supporting steps that feed a shared circle as carefully as the code behind it; and, where chains do not converge, fall back to ordering findings individually. All counts are of source-supported candidate routes in the ledger, and ring assignments follow the rule in Section 3.4. They locate where a defect would matter most; they do not show that any route has been exploited.

7. Limitations and next evidence step

This selected corpus cannot estimate chain prevalence across production software or establish how organisations prioritise review. Advisory/version matches and merged finding rows may be imperfect. The ledgers contain route variants, free-text outcomes and conditional assumptions; their B2 Major or B1 Critical labels need confirmation against exact resources and affected populations. We reviewed selected code handoffs and 54 exact-ID path joins, not every possible capability-based composition, and did not run the applications. A source citation locates evidence; it does not by itself establish a capability transfer.

The next evidence step is an adjudicated case set from the six changed forks. Each case should show the pinned revision, named fix, before/after code, remaining path, external conditions, execution environment and source-review judgement. Include both supported chains and failures such as Lumen's patched shared sink or a missing victim-session handoff. This resolves the current inventory into defensible outcome profiles without requiring live exploits.

8. Conclusion

Securing an application's outer entry points does not settle the risk deeper in its code. Weaknesses can compose across ordinary features, identity checks and privileged operations; independent controls at those handoffs are the code-level defence in depth that matters. Rings of Risk expresses this risk per protected outcome, showing consequence, entry conditions, distinct routes and the fixes needed to interrupt known paths. Across 15 OSS applications, selective hardening in six local forks leaves a substantial route inventory while also breaking chains that share patched code. The remaining source paths need outcome-level adjudication, but they already show why counting fixed findings alone cannot answer what remains reachable.

References

1. FIRST. CVSS v4.0 User Guide.
2. FIRST. EPSS FAQ.
3. CISA. Known Exploited Vulnerabilities Catalog and SSVC Guide.
4. OWASP. Risk Rating Methodology, Top 10 and ASVS.
5. NIST. SP 800-30 Rev. 1: Guide for Conducting Risk Assessments.
6. FAIR Institute. FAIR Standard v3.0.
7. Tenable. Vulnerability Priority Rating and Attack Path.
8. Rapid7. Active Risk strategy.
9. Qualys. Understanding TruRisk Score.
10. Microsoft. Security Exposure Management attack paths.
11. Ou et al. MulVAL: A Logic-based Network Security Analyzer. USENIX Security, 2005.
12. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. 2024.
13. CVE-Bench: A Benchmark for AI Agents' Ability to Exploit Real-World Web Application Vulnerabilities. 2025.
14. AgentCyberRange: Benchmarking Frontier AI Systems in Realistic Cyber Ranges. 2026.
15. MITRE. Common Weakness Enumeration and ATT&CK.
16. CVE Program. Overview; NIST. NVD CVE FAQ.
17. Snyk. Create a fix strategy and reachable-vulnerability analysis.
18. Wiz. Cloud and AI Security Platform.
19. NIST. Cybersecurity Framework; OWASP. SAMM.
20. Shimizu and Hashimoto. Vulnerability Management Chaining. *IEEE Access*, 2026.
21. Jajodia, Noel and O'Berry. Topological Analysis of Network Attack Vulnerability. 2005.
22. Wang et al. k-Zero Day Safety: A Network Security Metric for Measuring the Risk of Unknown Vulnerabilities. *IEEE TDSC*, 2014.
23. Yamaguchi et al. Modeling and Discovering Vulnerabilities with Code Property Graphs. *IEEE S&P*, 2014.
24. Lee and Brumley. ExploitBench: A Capability Ladder Benchmark for LLM Cybersecurity Agents. 2026; Shi et al. VEX-Bench. 2026.
25. Praetorian. Exploit Chain Analysis. 2026; Cycode. Attack Chaining. 2026. Both are vendor accounts.